

Online Game Design

simi

March 2024

Contents

1	Introduction	5
1.1	What is a game?	9
1.1.1	Having fun	9
1.1.2	Why is game fun?	10
1.2	How games change us?	11
1.2.1	The issue of violence	11
1.2.2	The issue of addiction	11
1.3	What to avoid in games	12
1.3.1	What is a boring game	12
1.4	Flow	13
1.4.1	A taxonomy of fun	14
1.5	What games teach	15
1.5.1	What's not the game	15
1.6	Virtual words	16
2	The structure of game	17
2.1	Experience and emotion	18
2.1.1	Modeling	19
2.1.2	Focus	19
2.1.3	Empathy	20
2.1.4	Imagination	20
2.1.5	Motivations	20
2.2	Games are lasagna	21
2.2.1	Mental model	22
2.3	Formal element of game	22
2.3.1	Players	22
2.3.2	Objectives	23
2.3.3	Procedures	23
2.3.4	Rules	23
2.3.5	Resources	23
2.3.6	Confilct	24
2.3.7	Boundaries	24
2.3.8	Outcome	24
2.3.9	Formal elements of games	24
2.3.10	Engaging the player	24
2.4	Dramatic elements of videogames	25

2.4.1	Challenge	25
2.4.2	Play	25
2.4.3	Premise	26
2.4.4	Character	26
2.4.5	Story	26
2.5	System	26
3	Game Design Document	27
3.1	The stages of the game design process	30
3.1.1	Concept stage	30
3.1.2	Brainstorming methods	31
3.1.3	Concept stage	32
3.1.4	Elaboration stage	32
3.1.5	Tuning stage	33
4	Formal Elements	34
4.1	Player	35
4.1.1	Invitation to play	35
4.1.2	Number of players	35
4.1.3	Roles of players	35
4.1.4	Interaction pattern	36
4.2	Objectives	37
4.3	Procedures	40
4.4	Rules	40
4.5	Resources	43
4.6	Conflict	43
4.7	Boundaries	44
4.8	Outcome	44
4.9	Dramatic Elements	46
4.9.1	Challenge	46
4.9.2	Play	49
4.9.3	Premise	51
4.9.4	Character	52
5	System Dynamics	54
5.1	Properties	55
5.2	Behaviors	55
5.3	Relationships	55
5.4	System dynamics	55
5.5	Balancing your game	56
5.5.1	Game mechanics	56
5.5.2	Game balancing	56
5.5.3	Hints	60
5.5.4	Various game structures	60
5.5.5	Economies	61
5.5.6	Emergent systems: Natural	61
5.5.7	Interaction with systems	62

6	GD role	64
6.1	Role of GD	65
6.1.1	Skills	65
6.1.2	Warren Spector	67
6.2	Game design team roles	67
6.3	A player-centric design process	68
6.3.1	The duty to entertain	68
6.3.2	The duty of empathize	68
6.3.3	I'm my own typical player	68
6.3.4	The player is my opponent	68
6.3.5	Other stuff influencing game design	68
6.3.6	Play-centric design process	69
6.3.7	Playtesters	69
6.3.8	Iteration	70
7	Playtesting	71
7.1	Classifying players	73
7.1.1	Demographic classification	73
7.1.2	Psicographic classification of players	73
7.1.3	Players' expectations	74
7.1.4	Context	74
7.1.5	Pleasure	74
7.1.6	Personas	75
7.2	Playtesting	75
7.2.1	Iterative design	76
7.2.2	Playtesting questions	76
8	Infrastructure	78
8.1	What is this "online mode"?	79
8.2	The Dawn of Online Games	80
8.3	How to get money from the users	80
8.4	What do we use for it?	81
8.4.1	Another perspective	82
8.5	It's a SERVICE	82
8.6	What is a Game Engine?	83
8.6.1	Asset	84
8.6.2	Rule	84
8.7	Runtime architecture	85
8.8	Game Networking	86
8.9	Unity Networking	86
8.9.1	UNet Architectures Overview	86
8.9.2	Netcode Transport Architecture Overview	86
8.9.3	Rules of the games	87
8.9.4	What do we need to sync over the network?	87
8.9.5	Spawning Gameobjects	88
8.9.6	How do we set up a game	88
8.9.7	Scenes	88

8.9.8	Using networkManager	88
8.10	Online Gaming Infrastructures	88
8.10.1	Frontend and Backend	90
8.10.2	Game cluster	90
8.10.3	Cluster architecture	90
8.10.4	Sub-server	90
8.10.5	Why clustering?	90
8.10.6	Workload	91
8.10.7	Modern Age Clustering	91
8.10.8	High availability	91
8.10.9	Business continuity	92
8.10.10	Make or buy?	92
9	Games and Network Interactivity	93
9.1	Network Quality of service	94
9.1.1	For UX	94
9.1.2	Origin	94
9.1.3	Today	94
9.1.4	How to?	95
9.1.5	Let's Face It	95
9.1.6	Overprovisioning is NOT the answer	95
9.1.7	QoS Today	96
9.1.8	It is NOT like that	97
9.1.9	Long Term Bandwidth Usage	97
9.2	Latency and Gaming	97
9.2.1	5G	98
9.3	How we deal with delay in games	98
9.3.1	How to obtain a uniform inter-arrival	98

Chapter 1

Introduction

Games must be taken very serious because they are very complex and mix together much sciences, they are multidimensional:

- computer science
- math
- physics
- design
- Cognitive psychology: how people react to stimulus
- Environmental design, we have to know basic knowledge how to structure a convincing ambient
- Storytelling
- Economy
- Visual art
- Music

Usually the game designer is also the team leader. In some cases we can say that games are arts and science.

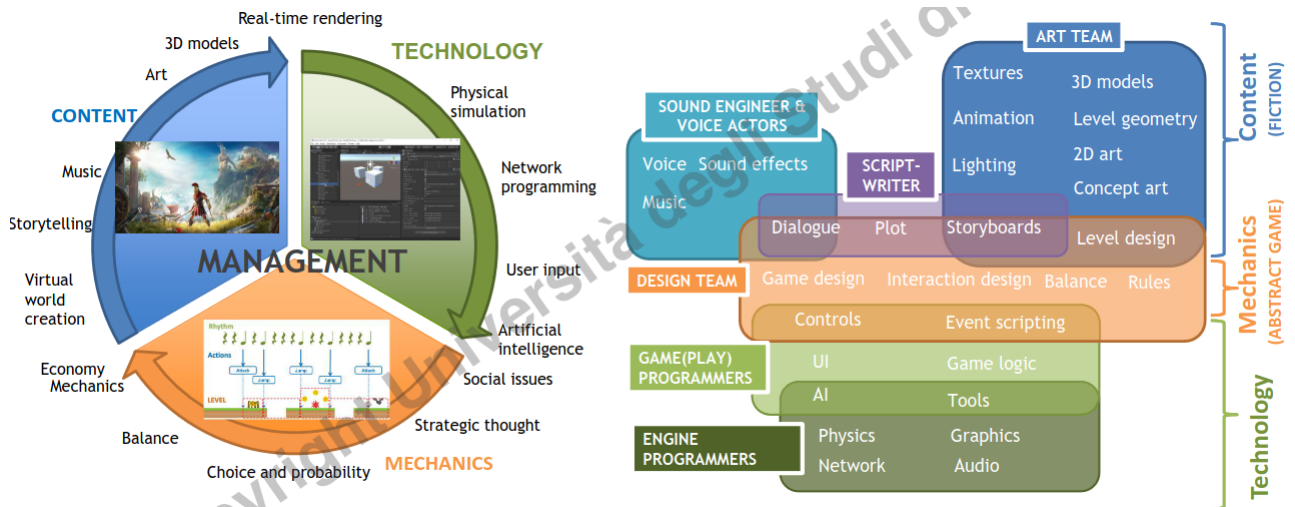
To create a video game we have 3 areas:

- Content: level design, economy, virtual world creation, storytelling, music, art, 3d models
- Mechanics: game design, social issues (how to make people interact in order to create guilds), strategic thought, choice and probability, balance, mechanics
- Technology: network programming, physical simulations

We can map this in the different professional profile in big company that develop games:

- Art team
- sound and voice actors
- script writer
- design team
- game(play) programmers
- engine programmers

There are some aspect that are done jointly by different teams.



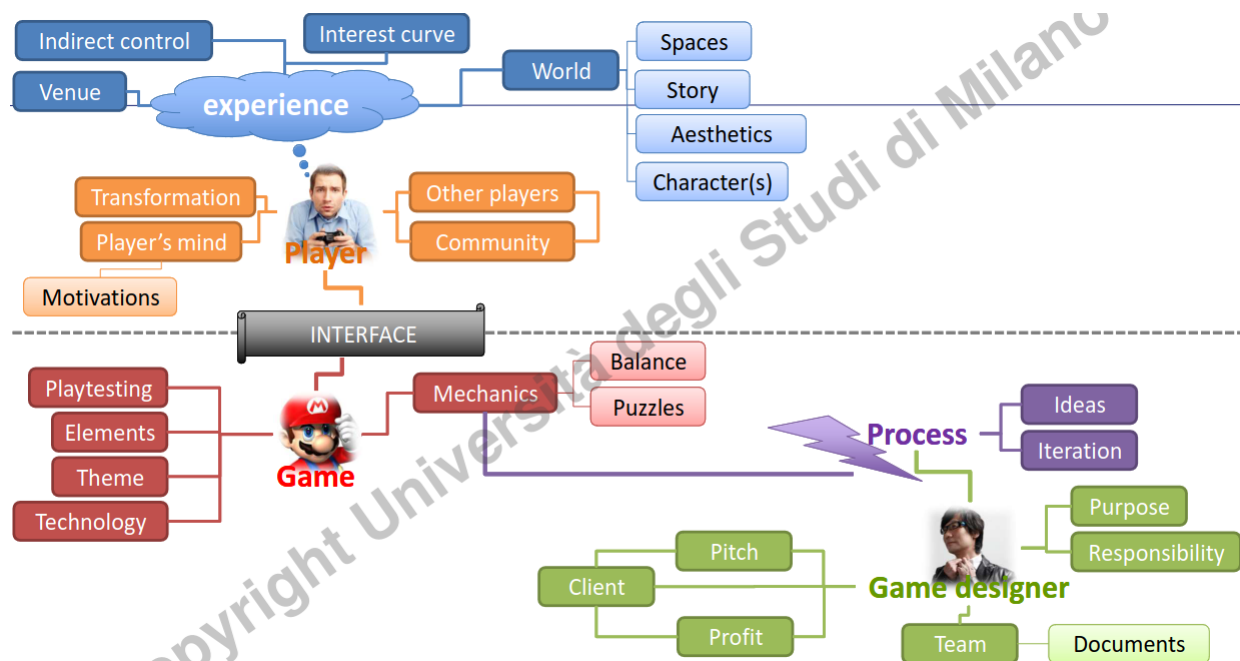
There are some knowledge useful for game design:

- Animation: understanding powers and limits of character animation opens the door for clever game ideas
- Anthropology: we are designing a game for someone, we have a audience and we have to know what the audience want and do
- Architecture: creates or cities or worlds (not only building)
- Brainstorming: we must know some technique to generate new ideas
- Business: we want to earn money from the games
- Cinematography: there are many techniques applied to video games that are taken to cinematography
- Communication: we have to coordinate the teams composed by different people with different background
- Creative writing: we have to create world
- Economics
- Engineering (compute science)
- Games: if we want to create games we must have huge knowledge of any types of games
- History: historical settings, Ubisoft had hired historical to design the different environment for the different level of Assassin's Creed
- Management: manage teams
- Maths: probability, risk analysis, scoring systems ...

- Music: needs for immersivity
- Psychology: the goal is making a human being happy
- Public speaking: we may have the necessity to present our game to convince someone that our idea must be developed
- Sound design
- Technical writing
- Visual arts
- Listening: we're creating, theoretically, something that will make someone happy and take the judgment from the user is important, and also taking ideas from the other element of the teams could be crucial.

Games are like lasagna, is not the ingredient that not make a good plate but is how they are mixed up.

The GD is like a chef that pick the ingredients and prepares the meal. It's important being able to create the perfect mix.



We have a diagram the has the different aspect of the game, in one side we have the GD that develop the game and in the other side we have the players. The game design work with a team for which prepare documents, he has to guarantee the company some profit, convince someone of the idea.

Each players are different and each has their own motivation to play game, while playing they transform the idea about the game, maybe they will discuss about the game in community, and overall this is giving an experience. While we are playing video game we are living an experience, depends on may factor, the world that we have build, how interested is the player, indirect control, venue (context in which I'm playing).

There are a lot of variable (in part we can control and in part no) that influence how our game will be.

Obviously the first thing that we want to know is how the player expect from the game.

Games are experiences, this means that we want is make people happy. The GD cares about the EXPERIENCE, not about the technology or some technical issues.

The game is not the experience, it enables the experience.

1.1 What is a game?

It's not easy to give a definition, we have a set of definition:

- Activity which voluntary, uncertain (we don't want to play game that we already know who will be the winner), unproductive, governed by rules, makebelive
- Free activity outside "ordinary" live
- A game is a rule-based formal system with a variable and quantifiable outcome, where different outcomes are assigned different values, the player exerts effort in order to influence the outcome, the player feels attached to the outcome, and the consequence of the activity are optional and negotiable
- Games are a subset of entertainment limited to conflicts in which players work to foil each other's goals, just one of many leaves off a tree that includes playthings, toys, challenges, stories, competitions, etc. Conflicts is important concept, in every games we have some form of conflicts
- One or more causally linked series of challenges in a simulated environment
- A series of meaningful choices
- A game is a system in which players engage in an artificial conflict, defined by rules, that results in a quantifiable outcome

None in these definition include fun, let's try to understand what fun means in games.

A child loves to play, no because it's easy, but because it's hard. For a small child there is no division between playing and learning and while we are learning something is fun.

1.1.1 Having fun

Is important to know what having fun is meaning. Fun in games arises out of mastery, when we start to control what is happening in game we feel good because we feel that we are able to play that game. It arises out of comprehension, how to solve a certain puzzle or overcome a difficult. It is the act of solving puzzles that makes games fun. With games, learning is the drug, during playing a game is crucial to figure out something, how to overcome a boss, how to win a fight. We're challenge to learn something.

So understanding what having fun means understand how our mind works.

As children we learned by trials and errors because we were trying to figure out patterns. We are exploiting one characteristic of our brain. Our brains use patterns, we are able to extract some characteristic to understand something, is something similar to a matching machine (pattern matching).

Our brain is very good at:

- fill in blank: we are not really "conscious", we do many things on autopilot, our brain know the patterns
- making assumptions: for example we don't notice our nose
- Cutting out the irrelevant
- Notice a lot more than we think
- Actively hides the real world for us: we usually draw the iconified version of an object, not the actual one. Hide the real world to recognize patterns, because is easier.

All this stuff is extremely important especially for visual artist.

In other words our brains also has limits:

- Maintain a limited number of (around 7) units of information in its short-term memory

but tries to overcome them:

- simplifies the information and remember the chunk of information, or the scheme of something

We rarely loo at the real work, we instead recognize something we have chunked and leave it at that.

1.1.2 Why is game fun?

We can do some assumption:

- is a representation of patterns, it's very similar to the way our brain works, our brain need to learn something that help us to play automatically, creating patterns
- Iconified representation of human experience that we can (safely!) practice with and learn patterns

Game can also implied boredom, because once our brain has nothing more to learn the game becomes boring.

Our brain is under drugs while we are playing because our brain release endorphins and dopamines while we are playing, while we don't have more to learn our brain stop to release there drugs and the game become boring.

1.2 How games change us?

Because the games teach us something in some way they have to change us. If the people were influenced by video games, then the majority of Facebook users would be farmers by now.

Can game be good for us:

- Emotional maintenance
- Connecting
- Exercise
- Education

Can be bad:

- issue of violence
- addiction

1.2.1 The issue of violence

Games often feature violent theme, but is also the same in stories, movies, comics etc. That we for us human something that include violence is automatically intriguing, because they are based on conflict. A violent act is a simple way to settle a conflict.

The presence of blood and realistic graphics have been target as violent and kids can't play. But it's only the surface because the structure of the games don't change. (mortal kombat vs virtua fighter)

It's real that players become desensitize of real world violence and gore, but it's the same with doctor, but usually the more you see through its metaphor and you are focused on:

- Pattern recognition
- Problem solving
- Solving puzzles

Average people are very good at distinguishing real and fantasy worlds.

1.2.2 The issue of addiction

Here the situation is more spicy. Self-destructive gameplay is nothing new:

- Gambling
- Bridge
- D&D and RPGs (paper and pen)

Addition in massive games is complex because different players have different motivation, like evade the real life unsatisfactory.

Avoid addiction

Ask to design games not addictive is like asking a cook to cook not good. We have to design games that fit in a good way into a well-balanced life.

GD have some sort of responsibility, because playing game change the brain.

1.3 What to avoid in games

We have to move away from the idea of gamers and try to think in prospective of average gamers.

One of the reason of which we get frustrating in the game is because we fail to see pattern in the game. Throught game testing we have to test that the pattern that we want to underline is clear. We have that player preciving noise.

Actually the more we practice the less we have to think about what we're doing, that's the base because games like tic tac toe becoming boring.

1.3.1 What is a boring game

When we meet noise we fail to see a pattern in it and we get frustrated and give up. Once we have figure out the pattern that we need to overcome the obstacle the game, that game becomes boring.

Games are puzzles when you are playing a game, you'll only play it until you master the pattern: once you've mastered it, the game becomes BORING.

Reasons for game to become boring:

- too easy: players groks the game pattern in few minutes (tic-tac-toe)
- too hard: player can't grasp any pattern. The game is just noise
- it's repetitive: pace of pattern variation in patter too slow
- too hard too fast: pace of variation too quick, we aren't giving to the player enough time to learn the pattern
- I beat it: player masters completely the game pattern
- unuseful to me: too many variation in the pattern. tons of possible permutations but not in the interest to the players, like a game in which we have to learning baseball results of the last decades

Boredom is the signal that we have failed something in the game. Because a good game teaches everything it has to offer before player stops playing (basically a game is a good game if the players want to play it until the end). But we are developing the game for a company, what happens in the prospective of the company if the game we develop is boring? if people'll quit the game we are developing contents that none will see, that are a useless costs.

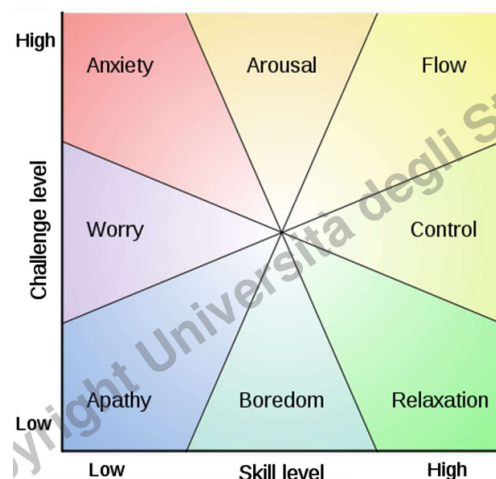
1.4 Flow

A psychology develop a definition of flow, he was trying to understand other characteristic when we are having a lot of fun: "the state you enter when you are experiencing absolute concentration on a task. When you are in absolute control, the challenges that come at you are met precisely by your skills". The idea is that the challenge that we are proposing to the players ideally perfect match the skill of the player. During the game my performance become better and better, this means that the game difficulty must escalate within the expertise of the player. Gamers must be aware that their skill is enough to beat the obstacle.

But how does it feel to be in "flow"?

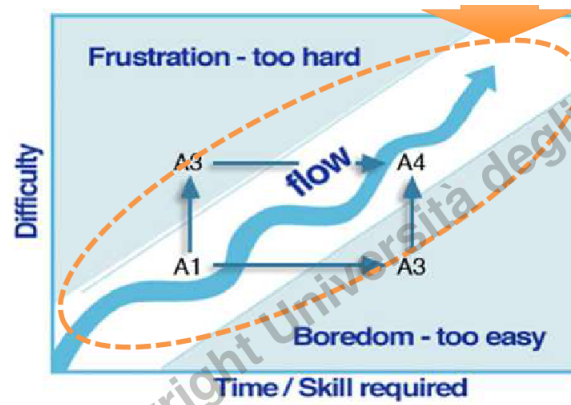
- completely involved in what we are doing, focused, concentrated
- sense of ecstasy of being outside everyday reality
- grate inner clarity: not only we know what we have to do but also we know what to do for performing well
- Knowing that the activity is doable, that our skills are adequate to the task
- A sense of serenity, no worries about oneself and feeling of growing beyond the boundaries of the ego
- Timelessness, thoroughly focused on the present, hours seem to pass by in minutes
- Intrinsic motivation, whatever produces flow become its own reward

We have a graph representation of the flow concept, in the horizontal layer we have the skill level and on the vertical axes the challenge level. This give us the idea when the game is boring or is frustrating.



Here we have the problem that we are all different, we are trying to trace a line for the flow for average player. So we have to focus not on a line but at list on a channel.

When people are in the flow they don't flow a straight line, but the prefer when the challenges have some variation. (sometimes over their skill levels and other time below).



There are several games that trying to resolve this problem letting the players to adjust the difficulty of the game by there self. For example flow is a game created for PS3 started by a PHD thesis.

1.4.1 A taxonomy of fun

Fun is the action of mastering a problem mentally:

- Aesthetic appreciation: not necessarily fun, but certainly enjoyable
- Visceral reaction: related to physical mastery of a problem
- Social status maneuvers: in multiplayer games we also have to be a good social designer

Social interactions

We have positive emotions related to interpersonal relations, are signals of pushing someone else up/down in the social ladder:

- Schadenfraude: when a rival fails at something
- Fiero: triumph for achieving a difficult task
- Naches: someone you mentor succeeds
- Kvell: bragging about something u mentor
- Grooming behaviours: intimacy
- Fedding other people: social signal in human societies

This are all positive emotion that we can leverage in multiplayer games trough mechanic that we have to design for this purpose.

1.5 What games teach

We have many types of game, and depending of the type of the game there are different things that we can learn.

- predict events
- to deal with power and status
- spatial relationship
- tool making
- classification
- explore possibility spaces
- exercise memory
- power
- team working
- social interaction
- etc.

All this type of skill that we learn through games are primitive skills, we're simply using in our days patterns that we were using in the beginning of humanity.

We have an art form on our hand that actually CHANGES BRAINS.

(Try diplomacy, it's a old but good game)

A game also can teach obsolete skills:

- Blind obedience
- Rigid hierarchies
- Binary thinking
- Force to solve problems
- Xenophobia

In time more and more we tend to increased the fidelity of simulation and not what we are simulating, for example jumping through the ages (campana to 3D platform).

1.5.1 What's not the game

Generally games are not abstract simulation, they adopt metaphors. Artwork has 2 main purpose:

- Teach us to see the math problem past the fiction
- Teach us to spot the same problem in different context

Games train you to IGNORE the fiction to find the underlying pattern.

1.6 Virtual words

Virtual words are computer generated phisical space but are also the strong component in cooperative games.

In virtual words we are developing a culture for the humanity, this parallel is exactly what happens in reality. When we visit the same space multiple time that space is not longer neutral for us, but we are developing culture (like same language and so one).

If we want to define virtual worlds, like wow, it's not a game but it's:

- It's a service
- It's a world
- It's a community

The Metaverse

Something that we have to know as a GD that there are games that are NFT based games. The idea behind this games is that I buy objects in the game as NFT and when I give up the game I can sell the objects.

Metaverse is not a recent tecnology, second life is a old game but it's basically the metaverse, and it stil exist.

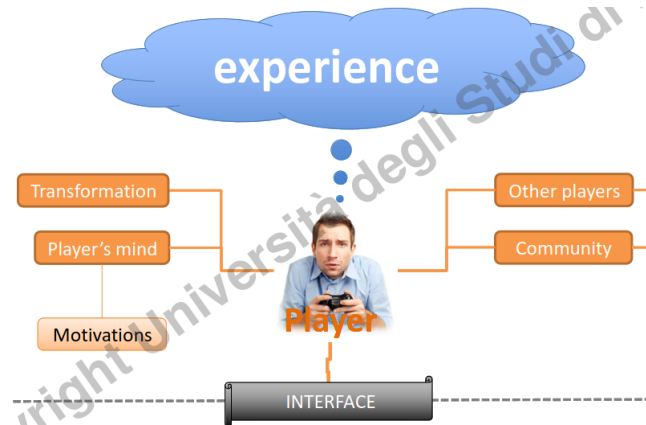
Chapter 2

The structure of game

We are warmly invite to install (and use) this small app "art of game design: lenses". It help us to check the design of our games.

Like we said games enables an experience. The gamers are moved by many reason to play, all this aspect are not neutral for the experience.

For the moment we focus on the side of the player, because the game is not the experience, it enables the experience! but we need to recreate the essence of the experience.



2.1 Experience and emotion

People may forget what you said, but they'll never forget how you made them feel. While we are design a game we have to ask our self:

- What emotions would we like my player experience? Why?
- What emotions are players (including me) having when they play now? Why?
- How can I bridge the gap between the emotions players are having and the emotions I'd like theme to have?

Think of reproducing the real experience of snowball fight in a game. We don't need to replicate exactly the same experience but we have to catch the essence of the experience:

- There was so much snow
- we played right in the street
- it was soo cold, but sunny with a blue sky
- kids were anywhere

As a GD our goal is to figure out the essential element that really define the experience, we want to create and find ways to make them part of our game design.

In the example of snowball fight:

- use artwork: puff of breath, shivering animations

- Use game rules: when hand get too cold, character must wear gloves and has a penalty

An other example is the game of James Bond, in which we can use hero points in risky situations to alter dice rolls. This recreate the supremacy of 007 in difficult situation.

Also we have a problem, we are creating experiences, but this experiences are living in the brain of someone else. There are several aspect that are in common to the humans brain, there are 4 principal mental abilities:

- Modeling
- Focus
- Empathy
- Imagination

2.1.1 Modeling

Reality is very complex so our brain simplifies it using some models of the reality. This process of modeling reality is automatic. Our mind do not deal with reality but with models of reality. So consciousness is an illusion that our internal experiences are reality, when in truth they are imperfect simulations of something we may never truly understand.

The example is that the colors is our approximation of electromagnetic spectrum.

Snoopy match our internal model of a dog. If we are using a graphic like this one we are relaxing the brain of the players.

There are many illusion that we simply cannot process. A good GD should understand how this illusion is formed in the players mind to create experience that feels as real as reality. The illusion of snow ball fight.

2.1.2 Focus

Our brain makes sense of the world by focusing its attention selectively. We focus on something ignoring the rest.

What we focus on, in the game, is determined by a mix of imperfect simulations of something we may never truly understand. A game should be an experience that holds the player's focus as long and intensely as possible in the FLOW.

People love repeating a cycle of:

- Increasing challenge
- Reward
- More power
- Easier period of less challenge
- Repeat

If we focus on the lens of flow it says to focus on solve some questions. But the real important thing is that flow is hard to test because we won't see it in 10' game play. It's easy to miss, we need to recognize the flow.

Once a player is in the flow observe her closely to find the crucial moment when she leaves the flow and understand why to improve your game.

2.1.3 Empathy

Humans are very good at projecting their selves into the place of others (even with animals, Disney). Brain does this using mental models:

- Cinema
- Literature
- Graphic art

Example of ICO, this game is based on empathy, because we have to save a girl that's more an obstacle than an help for us (prequel of shadow of the colossus).

Games are about problem solving, emphatic projection is useful. If I can image myself in the place or another, I can make better decision. For example in a Star Wars game, if I play using Darth Vader I usually make decision that in my opinion also Darth Vader would also make.

A game is interactive so the player not only has some feeling on the character, but also to influence the decision making process.

2.1.4 Imagination

Our ability to filling the blanks. I refer to the everyday imagination that every person uses for:

- Communication
- Problem solving

We are imagining small details that not necessarily has be thought.

We don't need to tell every small details in game, players will fill in the rest. Art helps in suggesting what you should show the player and what you should leave to her imagination. Also physic in game can help to fill the blank, in a car racing game we have to simulate physic enough to be realistic, not like a simulator.

2.1.5 Motivations

The goal of ANY entertainment form is taking something really UNIMPORTANT and making it SEEMS IMPORTANT. GDs should have a special insight in human MOTIVATIONS, to understand what can motivate someone to involve in story.

Marketing studies can help in this. This pyramid was used in many games, in the base we have the basic needs for human, we can move on the top of pyramid if the previous needs are satisfied.



Lets try to map games to field of pyramid:

- Level 4: achivements & mastery
- Level 3: multiplaier games
- Level 1 to 5: Minecraft (from gathering resouces and find shelter to multiplayer abou mastery and creativity...)

We can use these needs to thinking about what basic human deeds our game fulfills.

By the way the 4th level of hierarchy is intimately connected to games and to the need of being judged, in the other way people hate to be judged unfairly. Also the game can judge the player, for example assign badge.

Motivations can be intrinsic or extrinsic, an intrinsic motivation is playing because the game is fun, an extrinsic motivation is playing because I've payed for the game.

Do your player want to avoid punishment/shame or to seek reward/fun? that 2 are different approach to develop a game, we can also use both. Is up to use to choose which is the best solution.

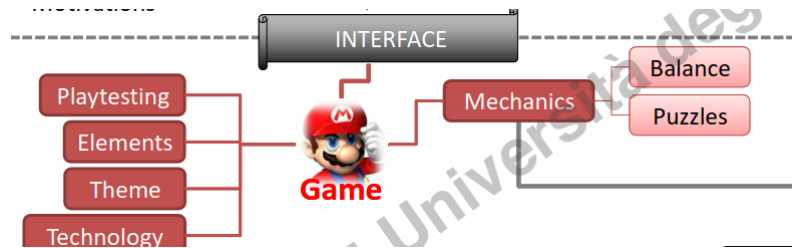
Novelty

Novelty is a POWERFUL motivation (humans are explorer). Novelty makes you think in a whole new way. Something that is really new that ask to think us in a new way (like portal) it's something that intrigue player because is really different. Be aware of too much novelty, successful game is a mix of novel and familiar.

2.2 Games are lasagna

Advice to game designer: play many games. Play them hardcore. Go beyond your own insight. Just watching someone play a game can teach you a great deal about game design.

There are different type of games, but they are all games... hende ...which are the common traits?



Which are the similarity between scala 40 and Doom? They are completely different game, but which are the similarity?

Try using chess, if we think about chess we have very abstract simulation of the reality. While I'm playing chess apparently I'm playing a war but I know that is a fiction. Basically for each games we have a formal structure and some metaphor covering the structure that provide the engagement of the game.

We can distinguish 3 level of abstraction:

- Fiction: is the story the player is told, the rationale for the game worlds. It does not affect the gameplay (chess could be a space war). It draws player into ghe game and halp to form a mental model
- Mental model: is the set of rules and data that the player infers and uses to make decision. It's the game the player is really playing.
- Abstract game: is defined by the real rules (not those perceived by the player)

Not necessarily the rules model that we (like gamers) have in mind match with the real set of rules. Like monopoly, because in monopoly we can build a house in every moment not only in our turn.

2.2.1 Mental model

Mental models are often partially incorrect, fiction could be so compelling to mislead (on purpose) the player.

An incorrect mental model might make a game more attractive than e correct one. Often the goal of a game is not to win as efficiently as possible, but enjoy the process of playing.

2.3 Formal element of game

2.3.1 Players

The first element are players beacuse games are experiences designed for players and require active participation by their consumers.

Players:

- Participate voluntary
- Both partake an consume the entertainment

- Voluntary accept rules and constraint. Lusury attitude is the curious state of affairs wherein one adopts rules which require on to employ worse rather than better means for reaching an end.

2.3.2 Objectives

The second formal aspect are objectives. Games lay out specific goals for the players! Objectives:

- Not present in other form of entertainment
- key elements: without them the experience loses much of its structure
- BTW: our involvement into the game is measured by our need to work towards the goal. The more we are involved in a game the more we want to reach the goal.

2.3.3 Procedures

The third aspect are procedures. Game have accurate description of what player can do to achieve the goals: rules allow specific actions/methods of play!

Procedures guide players behaviour, creating interaction that would probably never take place outside the game. Ex in Magic, if I need a certain card I simply cannot look through the deck for it, even if this would be much more efficient.

2.3.4 Rules

We also have rules:

- Define game objects and concepts (ex: deck of cards, draw pile ...)
- Limit player behaviour and forbid specific events: ex. you cannot use rifle bullet with a plasma gun

Procedures and rules work together creating AUTHORITY that stems from an implicit agreement by the players to submit.

Games are experience with rules that define object and principles and limit behaviour. Rules are key structural elements (that must be respected). Try to describe a game with NO rules, it is impossible.

2.3.5 Resources

Are object that have a value for the player, because they help to reach the final goal. They usually are made scarce by the designer. Finding and managing resources is key part of many games.

Value of resource is made by the combination of its scarcity and utility.

2.3.6 Conflct

The goals should be reached also if rules and procedures making this process spicy. Rules and procedures can also make players work against each other (or against the game system)

This generate CONFLICT, that can configure in different way.

2.3.7 Boundaries

Object and rules apply in the game, not in real life. Crate BOUNDARIES, that can be:

- Physical: create a 3D videogame space
- Conceptual: social agreement in GoFish

This create a Magic Circle, that is a temporary world where the rules of the game apply, rather than the rules of ordinary world. The playground is marked off either ideally or materially (arena, card deck, court of justice, temple, ...).

2.3.8 Outcome

Objective are not the outcome. We have the goals but the outcome could be different. The outcome:

- is uncertain, if we know that I'll win this wont be fun
- is measurable and unequal (a winner, looser, different scores)

Uncertainty is central for players, it's a key factor for a game designer. If a player can anticipate the results, he give up playing because it is boring.

2.3.9 Formal elements of games

We've examined elements that all games share. They provide a STRUCTURE to the game.

This taxonomy is not graved in stone, it aims at providing:

- Conceptual tools
- Shared vocabulary
- Contextualisation

Formal elements alone can provide an intriguing game?

2.3.10 Engaging the player

Good entertainment moves us intellectually and emotionally, different players like different drivers of engagement.

2.4 Dramatic elements of videogames

2.4.1 Challenge

As we said games intrinsically create conflict and the player trying to solve the conflict in his favour. Conflict CHALLENGES players and generates tension:

- Rising: achievement, ..., frustration
- Challenge flat/falling: boredom

Balancing emotional response to the amount of challenge is a key to keep players engaged.

2.4.2 Play

Play in itself is not a game, as a matter of fact play is free movement within a more rigid structure.

Even the more rigid game system can provide opportunities for players to use imagination, fantasy, inspiration, social skills etc to achieve goals.

Play can be:

- Serious: chess tournament
- Charged and aggressive (DOTA tournament)
- An outlet for fantasy (City of heroes, WoW, Star Wars Galaxies, Minecraft ...)

So remember that we can add some aspect of play in our game. And also remember to design the game for the type of players that we have in our game and also leaving a bit of space for free-play.

Also on play we can distinguish 4 type of plays based on the level of interaction:

- Story: no interaction. Cannot be manipulated by players.
- Toys: they can be manipulated by we don't have a fixed goal here
- Puzzle: are rule based system and the goal is to find the solution
- Games: are rule based system the goal is to win (we have a win condition)

Also different type of plays can be mixed to imagine new challenges / intrigue the players:

- Games includes some puzzles
- Puzzles that include a toylike construction set
- toy that players make puzzle-like through their goals
- story told partly through puzzle

2.4.3 Premise

Is something that create the context for the game, it always exists. It makes easier for players to contextualize choices. It's a powerful tool to involve players emotionally.

2.4.4 Character

In the traditional storytelling are agents through which stories are told. In the games are vessel for participation through the story.

They are a powerful tools for player engagement.

2.4.5 Story

In some games we have story to engage players emotionally. Note that premise and story are not the same, because story has narrative qualities and unfolds during the game (otherwise premise could be self-contained).

Remember that story telling game is one of the most difficult game to design, because the story have to be a quality story.

We can also provide some actions in the game.

2.5 System

Games are systems, so they are compose by group of interrelated elements that work together to form a complex whole. There are different element that are connected. It means that is very hard to predict what will happen when the game is played simply by analysing its element. We need a play testing for correctly test the game, that's because games are systems.

Chapter 3

Game Design Document

Creating a great Design Document is important. Usually GDD should contain every aspect of the game that is usefully for every component for the development team (music, combat system...).

A Game Designer must be able to create a well structured design document, it's not something strange because also other categories of professional must create design document like movie maker and so on.

There are several document that we have to create, depends on the stage of development that we are and different document become important:

- Concept document: we have to present the idea of our game to someone, like a client or a big company. usually in a big company a game is develop by different teams and a document to stick with is important. This document is useful to convince someone that the game is a good idea. It must contain:
 - Genre
 - Target audience
 - Description
 - Most compelling features: for which reason player'll choose our game respect to other?
 - Market information: which are the competitor
 - Cost and time to develop
- Game design document: it's like d&d manual, it gives all the detail to create the game. It describe the body and the structure of the entire project, with all the details, and the method by which each element will be implemented. It ensures that what is produced is what you want to produce
- Production documents:
 - Time-management charts (Gantt, PERT, and so on);
 - task database;
 - budget spreadsheet;
 - technical specifications;
 - Q/A database.

It implements the design document on time and within budget. Make sure that the development process is on track.

The GDD must be updated in time, we'll have a section of change log in which we put who, when, and why modify the GDD. There are 10 points of a successful design document:

- Describe not just the body, but the soul. Describe the aspect that are the metaphor of the game, like the outline of the game (behavioural traits of a character). One mistake is missing an inclusion the visual references of everything (not a word description but an image to what designer will create).

- Make it readable. The document must be read by artist and programmer.
- Prioritize: we have to prioritize features
- Get into the details
- Some things must be demonstrated: in certain cases is easy to demonstrate what we have in mind using pictures.
- Not just "what" but "how": for example how a weapon should be used by a character (sequence of moves)
- Provide alternatives: it could be technical (use Unity or Unreal) or something stick to gameplay (if this aspect is difficult to be implemented we have an alternative)
- Give it a life: change log, because we may want to roll back. It's also for the teams member, if something has been changed we have to see if our current work has effect
- Nobody should be able to say, "I DID IT THAT WAY BECAUSE I COULDN'T FIND ANY REFERENCE TO IT IN THE DOCUMENT."
- Deliver it in good condition

Possible mistake:

- If you cannot sum up your game in 2 sentence you don't have a game. Remember that in video games is quite normal to describe the game in terms of crossover (my game is a crossover between Mario and GTA).
- Constraint for video games:
 - cost and platform
 - Target audience: how your player is
 - Rating (PEGI): consequence of target audience

Other problems:

- How many players?
- Clear statement of the (final) goal of the game?
- Key feature do not put the accent on what's really special in your game
- No mention of the player type
- Review your English

3.1 The stages of the game design process

Unless a project is small, it's impossible to create a complete design and code it up afterward! because we have this interliving phases, playtest and update GDD.

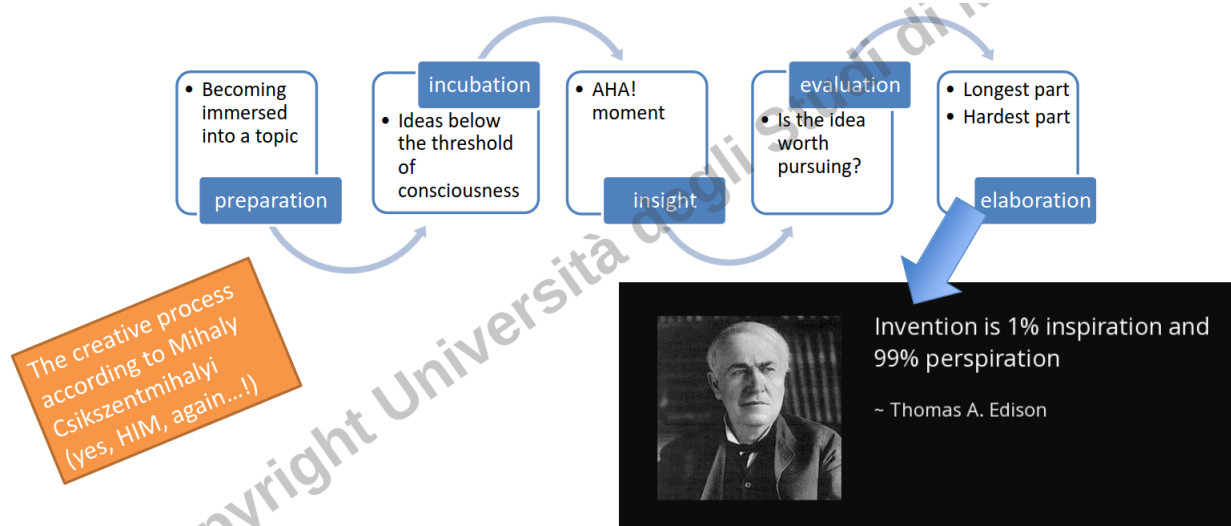
The design process can be split into three major phases:

- Concept stage
- Elaboration stage
- Tuning stage

3.1.1 Concept stage

The result of this stage are few things that are set in stone. Stuff decided here is so fundamental that any change later would wreak havoc on the development process (if you are building an hotel, you cannot change your mind and build a stadium ...).

Where do ideas come from?



Ideas come from where? try to have a lot of different interests, think of other games, they can inspire. Also look below the surface of your daily life, try to look everything that surround us in term of game system. Keeping a game journal could be very useful, because in some times the experiences with game that we play can be very useful.

There are some brainstorming technique for generating ideas:

Best practices	Possible methods/techniques
a. State a challenge b. No criticism c. Vary the method d. Playful environment e. Put it on the wall f. Go for lots of ideas g. Don't go too long	• List creation • Idea cards • Mind maps • Stream of consciousness • shout it out • Cut it up

State a challenge

Example, design a game:

- where players must make strong alliances and then they betray. We are focusing to create a specific player experience
- With a special role for parents to play with their children. We are focusing on the audience
- hat makes interesting use of only 1 button for control. Focus on technology

3.1.2 Brainstorming methods





3.1.3 Concept stage

- Get a concept: general idea of the game and why this would be a compelling experience
- Define an audience: who would enjoy the experience
- Determine the player's role: important for influencing both publisher and buyers
- Fulfill the team: define the essence of the experience you are designing

In big company step 1 and 2 should be reverted, they usually develop a game concept for a particular audience.

3.1.4 Elaboration stage

- Primary gameplay mode:
 - The mode in which the player spends the majority of her time
 - Includes: perspective, interaction model, challenges, actions available to overcome challenges
- Design the protagonist: the player identifies with it
- Define game world:
 - Real vs imaginary: we want something imaginary or something that is realistic? beware of reality, because we have to be much more precise
 - Look & feel
 - Dimension: physics, time, environment, emotions, ethic
- Design core mechanics: challenges are in place, start thinking which core mechs you need to implement

- Create additional (gamplay) modes: be careful, they require to rethink gameplay & mechs, creating a lot of stuff (assets...), a lot more playtesting etc.
- Design levels: most important milestone, while we have the first playable level we can start playtesting. It couldn't be digital, we can also have it on a paper, is much more cheaper.
- Write the story: must be integrated with the gameplay
- Build, test & iterate

3.1.5 Tuning stage

- Once elaboration stage has finished we have to lock the feature, nothing more should be added to the game. Usually in this stage we subtract something that is useful to have a polish game

Often this stage makes the difference between a good game and a really GREAT one! PS SCRUM management process could be adopted.

Chapter 4

Formal Elements

4.1 Player

4.1.1 Invitation to play

One the critical moment when a player decide to play is when he accept the invitation to join in the magic circle. In theatre we are waiting sipario open. In card game we wait that a friend invite us to play. In videogames we have different way to involve players, we can have physical object, we can have friends (community).

Start to think which is the mean to use to invite players to play our game.

4.1.2 Number of players

The number of players has grat impact on the game. We have different type of gameplay:

- 1 players: solitaire
- Exactly 2 player
- 2-4 players: labiring
- 4-8: monopoly
- tons: WoW

The number of players has direct link to the gameplay. Think careffuly the number of players that we want in our game.

4.1.3 Roles of players

Players have different play stile, we can find many classification. We use Barlte's players, we have 2 axis, interaction with game world or players and acting or interacting.



Depending on how we are designing our game you will attract different mix of types of players.

- Same role for all players
- More than one role: master mind
- A variety of roles
- Role-Playing
- Player-defined roles

Having different roles is not neutral in types of mechanics and type of players that we are designing.

4.1.4 Interaction pattern

Once we have set the number of players we have also to decide how they interact.

There are the main categories of the interaction pattern that we can have:



Single players vs. Game

It's very common pattern. Usually includes puzzles to generate conflict because there is not a real opponent. This pattern comes to mind when thinking of video games, but games have always been intrinsically multiplayer!

Multiple individual players vs. game

Multiple players compete against the game systems. There is not interaction between players. (bingo, candy crush). This pattern is not so common in video games.

This pattern works well for:

- Non-competitive players
- Gambling games

Players vs. Players

Two players compete (chess, tennis...). It works very well for:

- Strategic games
- Competitive players

Unilateral competition

Two or more players compete against one (bandiera, dodge ball). This pattern is highly undervalued, because in reality it is really good for:

- mixing cooperative and competitive gameplay
- Highly strategic

Multilateral competition

Three or more players compete. The pattern of many so called "multiplayer" videogame. In board game 3-6 is the ideal number.

Cooperative play

Two or more players cooperate against the game system. This pattern is undervalued, adopted often for children games. But it is possible to introduce innovation.

Team Competition

Two or more groups compete, we have teams, think about esports. Teams sprang up immediately upon the introduction of multiplayer/massively multiplayer.

4.2 Objectives

Give the idea to the player what they have to accomplish to complete the game. They are not neutral to the game, but they set the tone of the game. They affect:

- The formal system
- The dramatic system

32  The Lens of Goals 



Illustration by Zachary D. Coe

 To ensure the goals of your game are appropriate and well balanced, ask yourself these questions:

- What is the ultimate goal of my game?
- Is that goal clear to players?
- If there are a series of goals, do the players understand that?
- Are the different goals related to each other in a meaningful way?
- Are my goals concrete, achievable and rewarding?
- Do I have a good balance of short and long term goals?
- Do players have a chance to decide their own goals?



We said that games are about to achieving goals:

- You MUST be able to state your game's goal and state it clearly
- If you have SEQUENCE OF GOALS, you MUST STATE EACH and HOW THEY RELATE to one another
- The more easily player understand goals, the more he wants to play your game.

When a goal is set in the player's mind, it gives huge motivations.
Good games goal are:

- Concrete: clearly understandable
- Achievable: player knows she has a chance of achieving the goal
- Balanced: short & long term goal
- Rewarding: achieving alone it's ok, but it's better to make players proud of their achievement

Which types of reward the player is getting from achieving the goal. We can have different types of pleasures:

- Anticipation: only one can win
- Completion: reached all the goals
- Delight in an other's misfortune
- Gift giving
- Humor
- Possibility

- Pride in accomplishment
- Surprise
- Thrill
- Triumph over adversity
- Wonder



Generalize objective:

- Capture: take/destroy something of the opponent while not dying
- Chase: catch/elude and opponent
- Race: reach a goal before the others
- Alignment: arrange game pieces in a certain configuration
- Rescue or escape: get a defined unit to safety
- Forbidden act: get the competition to break the rules
- Construction: build/maintain/manage objects
- Exploration: explore games areas
- Solution: solve a problem before or more accurately than competitors
- Outwit: gain and use knowledge in a way that defeat competitors

If we are looking for new idea just try to mix up different types of objectives.

4.3 Procedures

Procedures are methods of play and actions that players can take to achieve the game objectives. We may think to procedures to who does what, where, when and how?

- How can use the procedure? one player? some players? all the players?
- What exactly does the player do?
- Where does the procedure occur? is the availability of the procedure limited by location?
- When does it take place? is it limit by turn, time or game state?
- How do players access the procedure? Directly by physical interaction? Indirectly through a controller/input device? By verbal command?

In board games procedure and rules are on paper. In video games procedure are integrated into the controls, rules are hidden.

There are some classes of procedures that are typically of video games:

- Starting action: how to start the game
- Progression of action: procedure after the starting
- Special actions: available depending on game state
- Resolving actions: bring the game to an end

Digital games can have:

- very complex game states, because the system take care of the rules for us, this is not possible in board game.
- hidden multifaced system procedure, responding to situations and player action

But high complexity may stem from very simple objects and procedure (chess).

4.4 Rules

Rules define game objects and allowable actions. They can be:

- Explained by players
- Explained in manuals
- Implicit in the game

When designing our rule we have to think of them relation to your players. And also think to state them very clearly.

Try to find some generalities for rules.

Defining object and concepts

Objects in games:

- Can be fabricated or based on ABSTRACTIONS of real world object
- Have a unique status and meaning defined as part of the game's rule set

Object in board game are defined explicitly by the rules. In video games can be made up by complex set of variables, managed by the system, that define their status. How player will learn about there objects?

Restricting actions

We need rules to:

- address potential loopholes in the game system
- produce delimitations
- avoid unbalancing the game

Determining effects

Rules can trigger effects based on certain circumstances. This:

- creates variation in the gameplay
- can get the gemplay on track: in single player you can die to force you to take care of your actions

Defining rules

Remember that:

- how you define the rules will be affected by the play enviornment
- Rules need to be clear
- Rules need do be intuitively grasped
- Rules should be fair and responsive
- The more complex the rule, the more demanding for players (the risk of discarding your game rises)

33  The Lens of Rules 



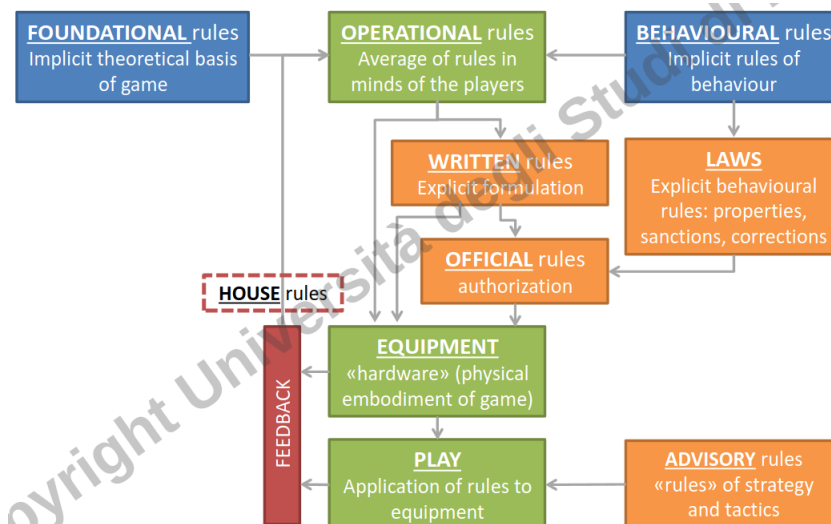
Illustration by Joshua Seaver

 To use this lens, look deep into your game until you can make out its most basic structure. Ask yourself these questions:

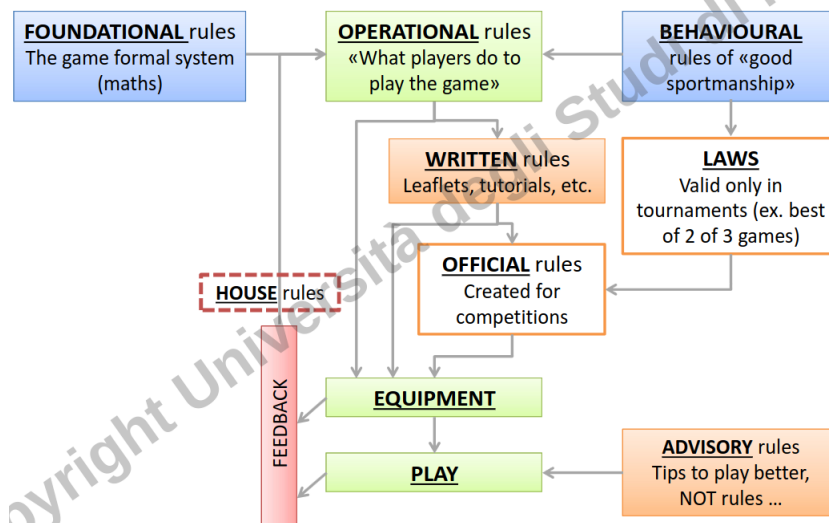
- What are the foundational rules of my game? How do these differ from the operational rules?
- Are there "laws" or "house rules" that are forming as the game develops? Should these be incorporated into my game directly?
- Are there different modes in my game? Do they make things simpler, or more complex? Would the game be better with more or less modes?
- Who enforces the rules?
- Are the rules easy to understand, or are they confusing? If confusing, should I change the rules or explain them more clearly?

We cannot sit down and write a set of rules, rules are arrived at gradually and experimentally (take notes while playtesting).

Written rules arrived toward the end of the design process. Part of the GD work is to be sure that rules cover every circumstance.



In this schema apparent games are set of rules, in reality games because are played by humans they have different rules. There is a set of behavioural rules, while we playing a game we have an etiquette to respect. The rules of the game are not the only rules to apply. Operational rules is the average set of rules that an average player have in mind (it's the mix of foundational and behavioural rules). This has an effect in how to use our equipment in the game (character). In the same way if we have written rules the operational rules that we have in mind effect in the way we interpret what we are read. We have also rules that are strategy suggested by other players.



4.5 Resources

Resources are assets. It is necessary to determine:

- Which they are
- How and when player can access them

Generalization of resources:

- Lives: in action games, the more the better
- Units: instead of live when player is represented by several object
- Health: helps to dramatize lives/units loss, requires ways to increase
- Currency: key for in-game economy
- Actions: moves/turns may be rare and scarce resources
- Power-ups: give boosts to players
- Inventory: used to collect (rare) objects that help players to reach the goal
- Special terrains: important in map-based games
- Time: periods of time may restrict players' actions, high dramatic effect.

4.6 Conflict

Conflict derives from restrictions to goal reaching posed by rules, procedures and situations:

- Due to inefficient procedure, players are forced to use particular skills, increasing sense of challenge and competition

Sources of conflict:

- Obstacles: more used in single players, can be physical or mental
- Opponents: other players
- Dilemmas: dilemma-based choices

4.7 Boundaries

Creates the magic circle. They can be conceptual (social agreement) and/or physical.

The designer must define boundaries and how to enter/exit them. Exception to magic circle is ARGs (Alternate Reality Games), mix real life and game.

4.8 Outcome

The outcome must be uncertain and be measurable and unequal.

No clear end state. Ending condition all the other games.

The final outcome is always related to:

- Players interaction pattern
- Nature of game objectives

People like to be judged fairly:

- Rewards are the way the game tells "you have done well"
- Rewards should fulfill the player's desire.

There are some common types of rewards:

- Praise
- Points
- Prolonged play
- Gateway
- Spectacle
- Expressions
- Powers
- Resources
- Status
- Completion of the game...!

You can mix rewards to satisfy different players:

- The more reward types the better
- Gradually increase rewards
- Variable rewards are more exciting than fixed ones



Why punishing players?

- Punishment creates endogenous value
- Taking risks is exciting
- Possible punishment increases challenge

Common types of punishment:

- Shaming
- Loss of points
- Shortened play (lose one life)
- Terminated play
- Setback (last checkpoint)
- Removal of powers
- Resource depletion

It's crucial that all punishments are for things that the player is able to understand and prevent. Even if it is true that some subgroups of players LOVE insanely challenging games (demon souls...)



4.9 Dramatic Elements

Analyze the aspect that are relevant in prospective for emotion of players.

Emotion is not the central point but emotion is something that add the possibility for player to express himself.

We said that formal elements create the game structure but dramatical elements add emotions to the game. They:

- Give context
- Integrate the formal elements in a meaningful experience (metaphores)
- Create a deeper sense of connection to the game

4.9.1 Challenge

Challenge is a task satisfying to complete, it creates a sense of accomplishment and enjoyment:

- Individualization: challenge determined by abilities of the player
- Dynamic: the game must adapt to increasing skills

If it is possible to adapt the difficulty of the game to the player experience is much more effectively. We want to the achieving the FLOW.

- A challenging activity that requires skill: having skills, but no guarantee of the outcome
- The merging of action and awareness: when all skills are needed to cope with challenge, attention absorbed by the activity

- Clear goals and feedback: we know what needs to be done and get immediate feedback
- Concentration on the task at hand: we are aware only about what is relevant here and now. Focus entirely on game.
- Paradox of control: I want to be in control, but if i give the complete control to the player the game will not be fun anymore. Control but be aware that the situation is not all under control.
- The loss of self-consciousness: too involved to care about protecting ego
- The transformation of time: perception of time is altered (videogames are addictive)

Experience becomes an end in itself, when most of the previous conditions are met, we enjoy the experience for itself.

Challenge and skills

Making a LIST of all the skills needed in your game is a very useful exercise. Listing skills can be very tricky.

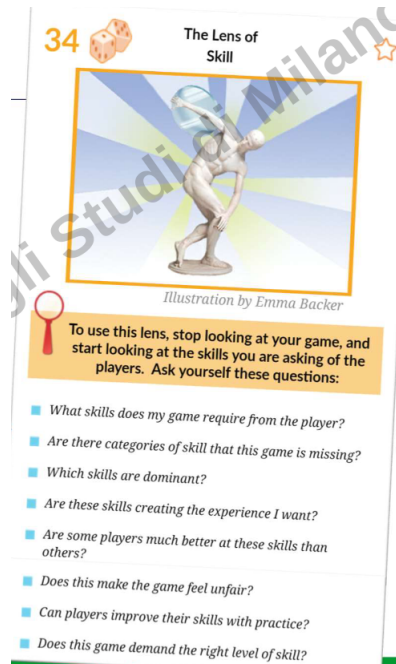
There are 3 main categories:

- Physical skills
- Mental skills
- Social skills

We can use the skill chart, we have different skills and we put an x on each level require a particular skill:

Skills / levels	01	02	03	04	05	06	07	08	09	10
Push trigger	x	x	x	x	x			x	x	x
Push block	x	x	x	x	x		x			x
Push creature		x	x	x			x	x		x
Open chest	x	x	x	x	x		x	x	x	x
Open lock				x			x			
Activate light	x	x	x	x	x					x
Object disappears	x	x	x	x	x	x	x	x	x	x
Object appears	x	x	x	x	x		x		x	x
Range attack		x	x	x	x	x	x	x	x	x
Clear opening			x	x	x		x	x	x	x
Clear floor			x	x	x		x		x	x
Clear wall			x		x				x	x
Cut vines/web					x	x	x	x	x	x
Cut rope					x	x	x		x	x
Cut creature					x	x	x			x
Super jump									x	x
X = present in level										

Note that not all the skills are necessary in every level, but mixing skills adds fun.

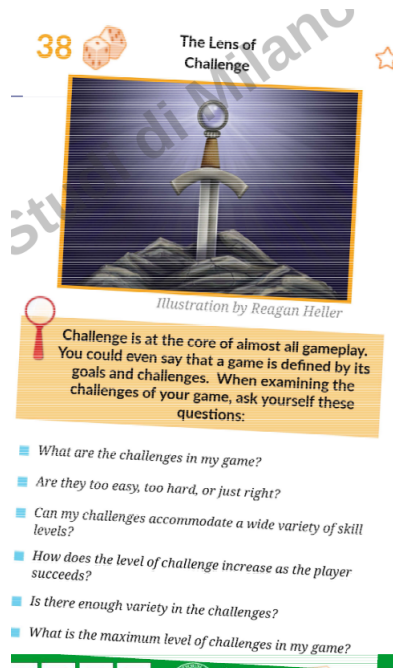


Exercising skills is joyful when skills are interesting and rewarding and the challenge keeps the player in the flow.

Challenge

We have to balance the challenge level for different players. We can use some common techniques:

- Increase difficulty with each success
- Let players get through easy part fast: skilled players should be allowed to dash through simple levels
- Create layers of challenge: giving stats
- Let players choose the difficulty level: one downside is that requires to develop and balance multiple versions of the game
- Playtest with a variety of players
- Give the losers a break: power ups in Mario Kart are more frequent for players behind



To define the level of challenge, remember:

- How many players do I want to be able to complete the game?
- Learning to play a game is a challenge
- Early success can do a lot to build player's confidence

4.9.2 Play

Play is the freedom of movement within a more rigid structure. It creates opportunities for personal expression. Play is a type of approach to an activity (a state of mind, not an action!).

Try to generalize some concepts:

- Nature of play
- Types of players
- Levels of engagement

If we think many games are built on top of toys. So we have to be sure that the toy is fun before building the game.



Nature of play

We can classify the games based on their nature:

PLAY	Free-form	Rule-based
Competitive	Foot racing, wrestling, ...	Boxing, billiard, football, chess, WarCraft III, ...
Make-believe	Masks, disguises, ...	Theatre, ... D&D
Chance-based	Counting-out rhymes, ...	Betting, lotteries, roulette, ...
Vertigo	Horse riding, dancing, ...	Skiing, mountaing walking, ...

Types of players

Different players are attracted by different games:

- Competitor
- Explorer
- Collector
- Achiever
- Lover

- Artist
- Director
- Storyteller
- Performer
- Craftsman

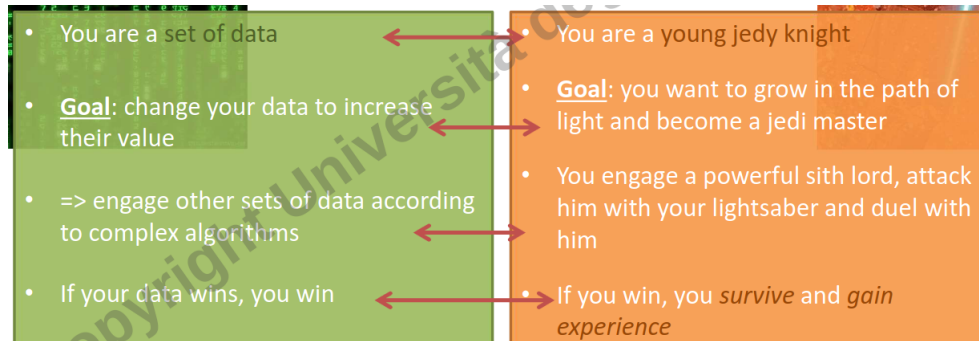
Level of engagement

Remember that not all players need to participate at the same level to have fun. Also spectators enjoy to watch esports event, having spectator have effect on emotional behaviour of the game.

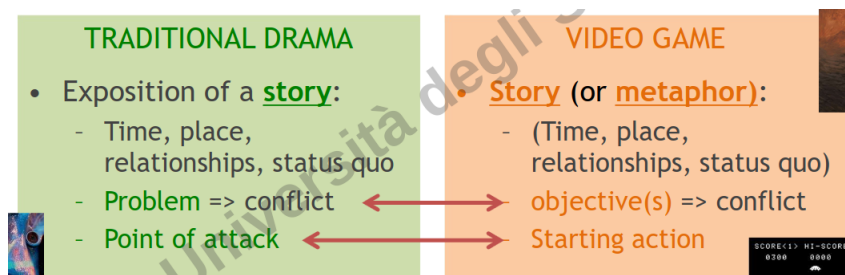
- Spectator
- Active player
- Transformational play (the game has effect in RL)

4.9.3 Premise

Same game, but same experience?



Premise establishes the action within a metaphor, connecting the players emotionally with the game.



4.9.4 Character

Characters are the agents through whose actions drama is told. In a game we are building the drama, because through our action the story will develop. Having an intriguing character is extremely important, because we are the hero.

In traditional drama the character are defined through the method of characterization, what they do, say, look like, other say about them. The same we can do when we are creating character for our game, we can start to ask to ourself:

- What does the character want and need?
- What does the player hope and fear

Remember that character in game che be defined as:

- Symbolic (american dream)
- Representative (etnich group)
- Hiscoric (real world figure)

We can put in the game a conflict between protagonist and antagonist. We can choose if the character is flat or evolve during the game.

Game characters should balance adency and empathy. Check this at every level of the game involving characters: is a predesigner? has a backstory? is player-created? can grow evole? can be customized?

Agency:

- Character as representation of the player in the game
- Can be utilitarian or include creativity

Empathy is the emotional attachments to the character.

There is different between characters and avatar. Characters pre designer, have a back story and basically we cannot change. On the other extreme we have character created by players and this type of character can change in time. Also characters can demonstrate some aspect of free will (AI controlled character). In the middle we have character that are controlled by the player but show certain personality. In the other extreme we have a character controlled by players.

Also remember that the possibility for narrative and immersivity grows in different way depending on how ive designed the character. There are 4 areas that influence how much immersivity i can supply to a character:

The more possibility we give to the player the more the game become immersive through the characters.

There are some peculiarity of the relationship between player and avatar, from total distinction to total identification (we project ourselves into other/things we control):

	Control	Customization	Things to do	Technè
<i>Super Mario Bros</i>	Yes	No	Few	No
<i>The SIMs</i>	Limited	Yes	Several	No
<i>WoW</i>	Yes	Yes	Quite a lot	Very limited
<i>SL</i>	Huge	Huge	Huge	Huge

+ Narrative & immersivity



Chapter 5

System Dynamics

5.1 Properties

Are qualities or attributes that define physical or conceptual aspects of objects (set of values that describe the object ex the stats). Descriptive data that can be essential to determining interactions of objects in a game system. The simplest types of game objects have very few properties, and those properties do not change based on gameplay (characters).

For example checkers have only three properties:

- Color: cannot change
- Location can change
- Type: can change

Three properties completely define the state of each checker within a game.

5.2 Behaviors

Potential action that an object might perform in a given state. The more potential behaviors an object has, the less predictable its actions within the system. Does this make the game fun? the addition of more potential behaviors tends to add choices and lessen the predictability of outcome in a game, not necessarily to increase fun.

5.3 Relationships

Can be expressed in various way (position, hierarchy...). How relationships are defined plays a large part in how the system develops.

Change in relationships can be determined by:

- Choices made by the players
- Combination of chance & rules

Note that if we have no relationships in our game we don't have a game but a collection.

5.4 System dynamics

A system requires that all elements are present for it to accomplish its goal(s). A system's components must typically be arranged in a specific way:

- If that arrangement is changed, the results of the interaction will change (imagine the effect of changing combat system)

Also remember that the consequence of what we've just said is that the systems are greater than the sum of their parts. Games can only be understood during play, when their dynamics become evident. So basically the GS have to craft a possibility space as best as possible. At the end they just do not know how each and every play of the game will go.

5.5 Balancing your game

The players want to be sure that the game is fair.

5.5.1 Game mechanics

Are the core of what a game truly is, the building blocks of the game, how the player is spending the majority of his time during the game.

We don't have universally agreed upon taxonomy, also because mechanics of each game are quite complex and difficult to disentangle, also they are strictly related to mental models so are difficult to describe.

Game dynamics emerge from mechanics and need to be balanced!

5.5.2 Game balancing

Balancing a game is like adjusting its elements until they deliver the experience you want!

It's the most artful part of GD:

- Understanding and managing the subtle nuances in the relationships among the game elements

We have several aspects to balance in games:

- Fairness
- Challenge and success: flow
- Meaningful choices: illuded that my choice produce effects
- Skill vs chance
- heads vs hands
- Short vs long

Balancing fairness

We have two sub-loops of games:

- Symmetrical games: all the players have equal resources and powers (chess, monopoly). Typically is easy to balance the game
- Asymmetrical games: opponents have different resources and abilities to:
 - Simulate real world
 - Give players another way to explore the game space
 - Personalize

To balancing the asymmetrical games we have to do a lot of play test. Halse of the development time is devoted to balancing. Balancing can only really begin once you have playable prototype. Balancing improves model, which is used to improve balancing, that improves model.

Fairness is a slipper subject. In some cases one side has an advantage, but the game is still fair:

- Player with unequal skills
- Disequalities justified by the game itself

The question to ask our self:



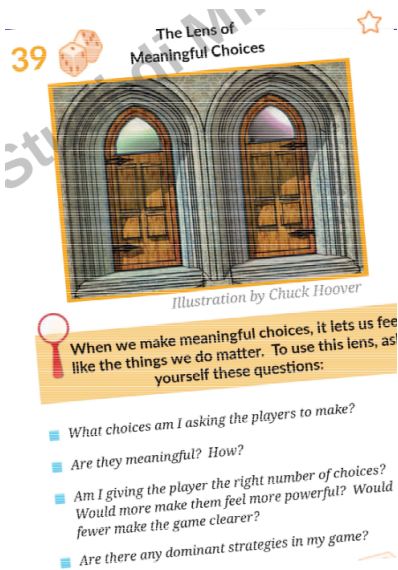
Meaningful choices

Good game is a game that gives players meaningful choices, they have an impact on the remaining of the game.

Beware of dominant strategies, once players discovered a dominant strategy the game is ruined.

So a good game is a game that force players to ask questions:

- Where should i go?
- How should I spend resources?
- What should I practice and try to perfect?
- How should I dress my character?



Meaningful choices

To determine the number of meaningful choices you need to figure out number and types of things players would like to do.

Also remember the lens of triangularity. Triangularity is when the player is in front of a choice, he has to decide if risk a lot for have high reward or risk low to get low reward.



Skill VS Chance

They are two opposing force. Too much chance negates the effect of player skill and vice versa:

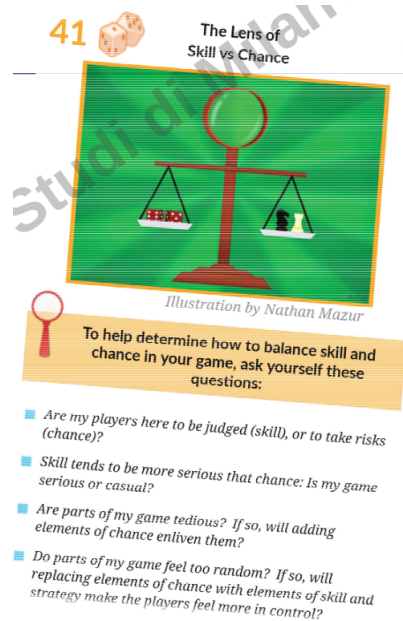
- Skill games: more competitive
- Chance games: more relaxed / casual

How to balance skill/chance highly depends on you audience. Also we have to take in consideration of the origin of players, it's clear with Germans, they are highly into into table-top games that are completely skill based and are competitive.

Common balancing method it to alternate the use of chance and skill during the game.

An addictive game generally ask 3 thing all the time:

- Exercise skills
- Take risk
- Work out a strategy



Head vs hands

How much of the game should involve doing a challenging physical activity and how much of it should involve thinking? There 2 aspect are not separated (action platformer, final boss...).

How to balance is highly depend on our audience.

Short vs Long

Is also important to balance the length of the gameplay:

- Too short: no chance to develop meaningful strategies
- Too long: boring

Length is determined by winning conditions.



5.5.3 Hints

Hints:

- State your problem clearly
- Doubling and halving
- Document your model
- Tune your model as you tune your game
- Plan to balance

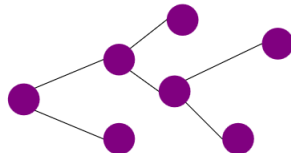
5.5.4 Various game structures

Different structure have an impact on the dynamic of the games:



Linear Flow

(Trivial Pursuit, side-scrollers, etc.)



Network or branching

(adventure games, etc.)



Objects or agents

(The Sims, etc.)

5.5.5 Economies

In some games we have an economy. The economy is an important structure often found in games: it affects one of the fundamental formal element (resources). In-game economies are generally severely controlled and generally only vaguely resemble real world economies. To have an economy we need:

- Items to exchange (resources)
- Agents of exchange (players, bank...)
- Methods of exchange (market, bartering...)
- Currency
- Prices (free, fixed ...)

Basic questions to design an in-game economy:

- Does the size of the economy grow?
- How is the supply of the currency controlled?
- How are prices set?
- Are there any restrictions on opportunities for trade?

	Amount of product	Money supply	Prices	Trading opport.
<i>Simple bartering</i> (Pit)	fixed	n/a	fixed	Not restricted
<i>Complex bartering</i> (Settlers of Catan)	Controlled growth	n/a	Market value with cap	Restricted by turn
<i>Simple market</i> (Monopoly)	fixed	Controlled growth	Market value	Not restricted
<i>Complex market</i> (Ultima Online, Everquest)	Controlled growth	Controlled growth	Market value with base	Not restricted
<i>Metaeconomy</i> (Magic)	Controlled growth	n/a	Market value	Not restricted

5.5.6 Emergent systems: Natural

Emergence: natural phenomenon, very simple rule sets, when put in motion can produce unpredictable results.

- 1 ant: lives with a very simple set of rules
- A colony of ants: a spontaneous intelligence emerges capable of very sophisticated behaviours

In video games the goal is to have a semirealistic behaviour of NPC. There are no pre-written scripts, but decision based on the context.

- Hale. NPCs governed by only 3 rules:
 - Perception of the world
 - State of the world
 - Emotion

5.5.7 Interaction with systems

Game are interactive they are design for interaction. We have some aspect to consider:

- Information: how much info do players have about the system state?
- Control:
 - What aspects of the system do player control?
 - How is that control structured?
- Feedback:
 - What type of feedback does the system gives to the players?
 - How does this affect the gameplay?

Information

We may have different type of information to give to the players:

- Open information structure: full disclosure of the game state (strategy game)
- Hidden information structure: game state hidden from players (guessing, bluffing, deceiving games)
- Mixed info structures (black-jack)
- Dynamically changing info structure: the amount of info varies dynamically during the game (fog of war)

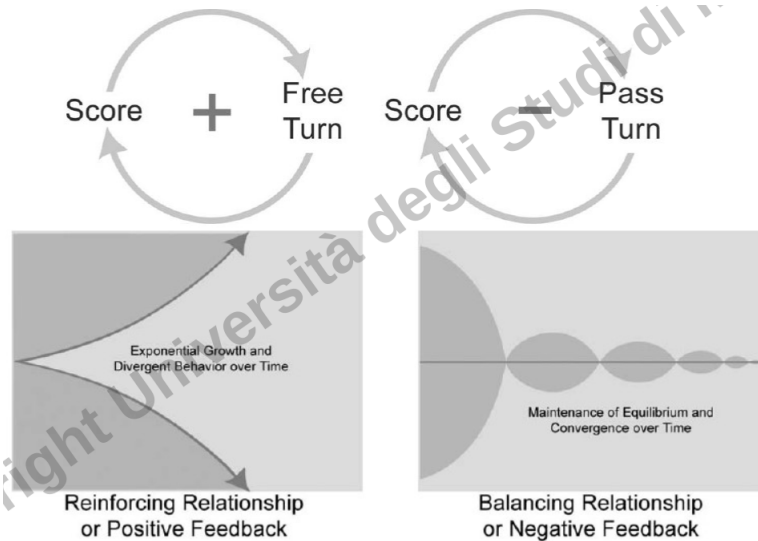
Controls

Also the type of control that we use (controller) have an impact in the design of the game. This is critical because the interface the player has is the top level interaction with the game.

Feedback

Feedback in systems implies a direct relationship between output and a change in the system. Many games use reinforcing loops to create satisfying risk/reward scenarios. Balancing relationships are used to keep the game from resolving too quickly.

This is another interesting aspect related to balancing the feedback. Can be a positive or negative feedback. If I score well we want to give a player a positive feedback, but if we are giving a reward to a player that does right we are increasing the distance between the skill's players and the no skill. The same happens with the negative feedback.



Chapter 6

GD role

Since there have been games there have been game designers! The history of games dates back to the beginning of human culture. We play to acquire skills (videogames are not exception). What makes games intriguing is NOT related to technology/medium, but to the EXPERIENCE of the players! The first step to become a GD is to ask ourself why we are playing certain games.

Game design is not a pure art because it is NOT primarily a means of aesthetic expression. It is not purely an act of engineering, since it is not bound by rigorous standard methods.

Goal of GD: entertain through play =, designing games requires both creativity and careful planning !!!

Game design is the process of:

- Imagining a game
- Defining the way it works
- Describing the elements that make up the game (conceptual, formal, dramatic, artistic, etc.)
- Transmitting info about the game to the team who will build it

ONE person must have the authority to make final decisions, and the other team members must acknowledge his/her authority.

6.1 Role of GD

Game designer is an advocate for the players (must look at the game through their eyes). This is too often underestimated, but it is the key to success.

GD envisions how a game will work when played. GD is responsible for planning everything necessary to create the game experience. GD plans the structural elements. Main task: make sure that when the game is delivered, it provides superior gameplay!

6.1.1 Skills

Great GD has a passion for creating playful situations, and is able to keep alive this passion. Has several skills useful for:

- Communication
- Teamwork
- Process
- Inspiration
- Becoming a better player
- Creativity

Communication

Must be able to communicate clearly and effectively with other people in your team in the game:

- Good language skills
- Crystal-clear vision
- Well-conceived presentation

And also become:

- a good listener
- an excellent compromise

This because video game production is an extremely intense multidisciplinary collaboration process:

- Computer scientists
- Illustrator
- Executive and business managers

They all speak different languages, and the FD is the universal translator.

Process

Production is a messy business, that may have disruptive effects on game. To avoid risks establish a GOOD PROCESS from the beginning.

Inspiration

A GD has a different way to look at the world, is able to see and analyze what he's seeing in terms of system and to find inspiration in common things

- Look at the world in terms of underlying systems
- Don't fall back on existing games for new ideas
- Examine a system, break it down and try to understand how it works

Becoming a better player

Does not mean get better skills but become a game LITERATE. We have to learn to play with CRITICAL attitude toward the underlying system and use this knowledge to create new games.

Creativity

Remember that everybody is creative in his own way.

6.1.2 Warren Spector

The education of a GD should be as broad as possible, and should include at least:

- Communication
- Behavioural psychology
- Programming
- Architecture
- Economics
- History
- Art
- Graphics
- Having played A LOT of different games
- Having made games

6.2 Game design team roles

We are focusing on the game design part of the team.

- Lead designer: the keeper of the vision, oversees the overall design and development process and evangelized team members
- Game designer(s):
 - Defines and documents how the game works
 - May conduct background research and data collection
 - Reports to lead designer
- Level designer/world builder: take components provided by game designer and builds levels/world
- User interface designer: often are usability experts
- Writer: create instructional and fictional contents
- Art director/lead artist:
 - Manages production of all visual assets
 - Creates the visual style of the game
- Audio director: the same of the art director for music/sounds

6.3 A player-centric design process

Is basically a philosophy of design in which the designer envisions a representative player of the game he is creating. Hence he has 2 duties:

- The duty to entertain
- The duty to empathize

While avoiding misconceptions:

- I am my own typical player
- The player is my opponent

6.3.1 The duty to entertain

The player spends time and money on your game so our first concern **MUST** be to be sure he enjoys himself. Entertaining players has a higher priority than expressing your creativity.

6.3.2 The duty of empathize

Put yourself in the shoes of your representative target player (a 6 years girl, a 75 grandpa, etc.) and imagine what it is like to play your game.

For **EVERY** design decision you **MUST** ask yourself how it meets the player's desires and preferences.

You need to think how the player will react to **EVERYTHING** (artwork, interface, gameplay, etc.).

6.3.3 I'm my own typical player

You **CANNOT** assume that players like what you like. You **MUST** learn to design what **THEY** like. Professionalism is just as important as passion!

6.3.4 The player is my opponent

Avoid the arcade games clichè: the designer job is not to create hard obstacles preventing the player to win the game. Model ok **ONLY** for games making money from players putting coins into a machine. Does not take into account players' motivations and/or interests. Ignores creativity (games that may not include obstacles, etc.).

6.3.5 Other stuff influencing game design

- Market-driven games: you can't make a brilliant game just by putting in all the most popular gameplays
- Designer-driven games: you can't ignore playtesting

- License-driven games: constraint creativity, but the resulting game must be as good as if it didn't have a license
- Technology-driven games: beware of not spending your time on the technology, instead of on making sure the game is enjoyable
- Art-driven games: Rare. Show off of artwork.

6.3.6 Play-centric design process

Having a solid process for developing is a CSF. Key features:

- Setting player experience goals
- Prototyping and play testing

Setting player experience goals

Goals for the type of experience that player will have do not describe the game, only the experience.

It's the first thing to have clear in mind:

- Ok for involving players in development from the very beginning
- helps focusing creative process
- helps getting inside the head of the players

Prototyping and play testing

Ideas should be prototyped and playtested early: build asap a playable version and a physical prototype of the core game dynamics to get instant feedback.

Usability aspects more and more relevant, especially for new, unexperienced players (VR).

To avoid costly mistake do not start production before having understood player experience and core game mechanics.

6.3.7 Playtesters

Play your game and give feedback on the experience. By watching other people playing your game, you can learn quite a lot!

Pay attention to what they are focused on, where they click/move the cursor when bored, what they tell you. Understanding playtest can be dangerous, since games are not one-way communication: you must build an experience, you are the host of a party, the interactive experience is fully realized only when the guests arrive.

Playtest is the heart of the design process.

6.3.8 Iteration

The game design process is some more that involve in that way:

- Experience goals set
- Idea conceived
- Idea formalized (prototyped)
- Idea tested against experience goals
- Result evaluated

Games are living systems that EVOLVE during the development cycle. Repeated testing and careful observations are extremely important.

Whole development process:

- Brainstorming:
 - Set experience goals
 - Create related concepts
 - Choose the top three
 - 1 page description
 -
- Physical prototype
- Presentation
- Software prototype: rough model of core gameplay (several prototypes about different aspects)
- Design document: collect info while prototyping and start writing a draft doc of every aspect of the game (design document)
- Production:
 - Verify with whole team that each aspect of the game is achievable and correctly described in the design doc
 - Create artwork and programming
 - test

- Quality assurance: verify aspects like accessibility and playtest again

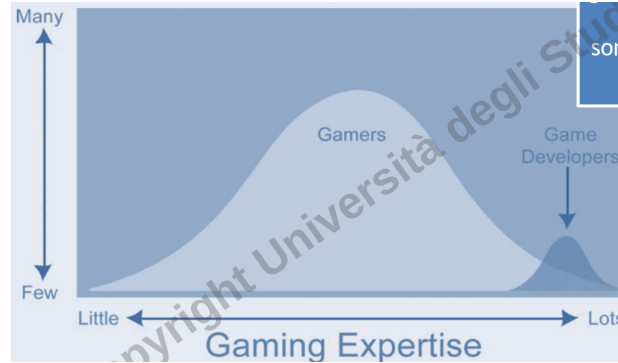
In reality some steps are compressed for time and money saving.

Chapter 7

Playtesting

The starting point must always been understand our target audience and design for them.

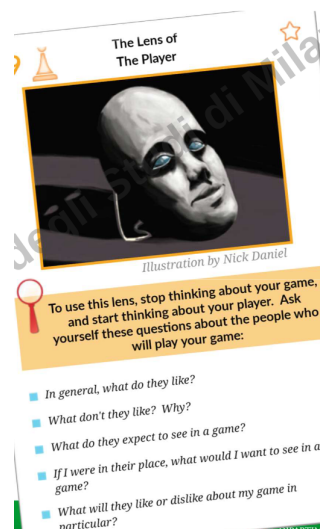
Game developers cannot make games that appeals to them, if they want to create something that the majority of gamers likes!!



So basically we have to design for a target audience. And as we said the first step for a good design is to know your target audience.

- Exploit every single occasion to talk with your potential players
- Never assume you know the ludic needs and expectations of your players (what you think may turn out to be wrong)
- Exploit interviews and observation to understand players
- Remember: players may be unaware of what they expect for a game (it's not simple to explain why we like a certain game)

To define your target audience ask yourself:



7.1 Classifying players

We need to classify players from demographic and sociological perspectives:

- Demographic taxonomy: age or geographic area
- Psicographic taxonomy
- Players' expectations
- Context

Based on these categories we can build some personas. personas are fictional players that represents some kind of players that we can have in our game.

7.1.1 Demographic classification

We are focusin on external factors (how they apprear):

- Age
- Gender
- Income
- Ethnicity
- Geographics
- Occupation
- Education

Some of these aspect are important, for example ethnicity because there are different aspect related to different culture that we have to understand. It's also useful for define the number of players that we'll have.

We can see an other type of classification, we can focus on age of the players. Age are very important in different types of design.

This type of classification is based on external factor but is more interesting in term of designing.

Beware also on stereotypes, be reliable of data!

7.1.2 Psicographic classification of players

Let's move on more internal factor of players. We have to try to understand which are the factor that influence the experience of playing. We have to study players lifestyle to understand how their activities, habits, interests and ideas influence their preferences.

There are many types of classification of players that we can find online, but in the game industry it is quite diffused the Bartle's classification.

Age (years)	Main characteristics (males and females)
0-3	Infant/toddler: interested in toys. <u>No</u> problem solving skills.
4-6	Preschooler: simple games played with adults (able to read and explain rules).
7-9	Kids: problem solving skills start to develop, more complex games.
10-13	Preteen/tween: very strong interests for specific game types (quite an obsession).
13-18	Teen: specific preferences and interests are evolving and settling (also gender-dependent).
18-24	Young adult: preferences & interests well defined, gradual reduction of parental control, gradual increase of income available.
25-35	Twenties & thirties: Players split into casual and hardcore.
35-50	Thirties & forties: main interest: career and family. More money available. Games should involve the family (consoles, etc.)
50+	Fifties & up: Plenty of spare time and desire for change: games with social component (golf, tennis, MMORPG, etc.)



7.1.3 Players' expectations

The expectations stem from culture, age, and they may be vary/evolve in time. Beware of:

- Cultural agreement
- Reaction to specific symbols
- Visual aspects
- Genre-specific conventional rules
- Player experience with similar games

If underevaluated, problems may arise.

7.1.4 Context

That's an other factor that influence a lot the experience of the players. Remember that people may change favourite game type according to:

- Psychological status (stressed, relaxed ...)
- Place: at home, commuting, with friends, on holiday

7.1.5 Pleasure

Pleasure derived from playing can vary (also according to the context):

- Anticipation
- Completion
- Delight in another's misfortune

- Gift giving
- Humor
- Possibility
- Pride in accomplishment
- Surprise
- Thrill
- Triumph over adversity
- Wonder

Starting from the type of emotion that we want to deliver we can change a little part of our game.

7.1.6 Personas

Once we have collected all these information we can obtain personas. The idea is to create archetypes of your players, as detailed as you can and design for them:

- Characteristic derived from demographic/sociological/cultural/context ... analysis
- Give them name and face
- AVOID average player

Reverse/complementary approach: define the profile of your non-player.

7.2 Playtesting

Something is something no so exciting. The goal of playtesting is to make clear to you that some of the decisions you were completely comfortable with are completely wrong.

Playtesting is NOT:

- GD & team play game and discuss features
- Rigorous test of the software
- Marketing executives looking playtesters and discussing sells
- Using data hooks

Playtesting is the most important activity for the game designer, and the least understood. Must be performed during the whole design process. In different moment of developing playtesting may have different purpose.

The goal of PT is gaining useful feedback to improve the overall game experience, the game should be internally complete, balanced, fun to play!.

7.2.1 Iterative design

GD MUST keep a strong relationship with players' need and perspective throughout the design and production process. Continual ITERATIVE process (playtest, evaluate, revise) guarantees that the game stays OK and that problems to solve get smaller and smaller.

7.2.2 Playtesting questions

- WHY
- WHO
- WHERE
- WHAT
- HOW

What data will I collect after the play session?



WHY?

Playtest is prototype of the game experience. You must playtest with specific goals in mind. We have to prepare this list in advance:

- Do men and women play differently?
- Do kids like my game better than adults?
- What strategies do players find on their own?

- Is level 3 too long?
- Are players ever frustrated?
- Which parts of the game are the least/most fun?

WHO?

playtesters MUST be chosen very carefully. If we want to playtest a casual game for mobile, appealing to adults commuters in a big city hires a group of teens leaving in a rural area is not a good idea. Represent our target audience.

Finding the ideal playtesters:

- Is someone who represent your target audience
 - Give more valuable feedback
 - Is able to compare your game to similar games
 - Knows what he likes
- How to find them?
 - Tap in your community, online posts, recruit among students...
- Fundamental characteristic:
 - Able to articulate enough to convey opinions (test in advance through quick interview)

In different phases of development process we want to involve different type of players. At the very beginning of the project is ok to test by ourself. At the very first stages is ok to test with our friends. But after we need to test with people we don't know that relates we honestly about the game problems!

Prototyping Stage	Playtest on Your Own	Playtest with Confidants	Playtest with Target Audience
1) Foundations	●		
2) Structure	●	●	
3) Formal Details			●
4) Refinement			●

Chapter 8

Infrastructure

In the infrastructures we'll talking about the technical part. We'll talk about:

- Introduction to online gaming
- Organization of supporting infrastructures
- Cloud support and game streaming. Cloud gaming is not always streaming games. Cloud is about virtualization of computational power and storage around the world. It can give us infinite computational power and data storage. We can pay what we consume, we use computing as a service. With this possibility we can finally create an online game with 1000 players together, that was impossible with a stand alone configuration.
- Issues of real-time communication for games: it's actually something that is very spike to handle. Like when we are on YouTube and when we start to see a video? how is that possible? How the network can be so responsive? Today this stuff are working because we are giving over provisioning of the network.
- How to plan your project

8.1 What is this "online mode"?

For many of us, it is just the bunch of additional functionalities we can use when your PC is connected to the network. But Telegram works in the same way, we have to attack purposes to these functionality.

Play online is not playing together, playing online is sharing an experience.

Split screen is playing together, not online. Because this is a different experience.

Still in a lan party is playing together in the same place.

The point is not a matter of playing online, it is a matter of being online! It's about the whole playing experience. We can use online feature to amplify the player experience and involent like:

- Global scoreboards: we have presence online
- Trophies: we have reputation -> also offline trophies
- Social networking: community

The idea is to provide all this tools in order to have a better experience with game.

There are companies out there focused not on games, but on online experience. These are experience providers, not games providers.

For example Roblox is escaping game definitions, because we have not scoring, teams and other stuff, but it is an online experience. The same thing happens with Minecraft, we are performing in a single point of the network, not attached to our profile. In game like OGame we are never offline, because something happens even when we are offline. In this case we have a life offline and an other online. LifeLine is a game for Android phone, we are chatting with a guy that landend alone in a planet and we have to suggest what to do. That kind of game try to overlap our phicial time to our virtual time. An other

important aspect in some online games is that we must create a team, otherwise we'll not do any progress in the game.

Creating an online game is about providing all the tool required to enjoy a shared experience (not only the infrastructure).

8.2 The Dawn of Online Games

13 years ago buying a game was something like going to a shop, buy a box that hold everything that was required to play game. Go back home and that's it, that was all the experience. Those time are gone, and are gone forever.

Today we download a software (sometimes for free) and then we pay for the online service. There are some games that ask to pay for the software and also for the online service (like WoW).

- Download: was a strong change in the distribution model. Years ago we have to ask someone to create a box for us, bring the box to a shop that'll sell for us. Today with download and online platform thinks are easy for new developer. It's cheaper, and we have illimited copies are always available. Today we may have infinite number of users in day 0, is our server capable to serve thousand of players in day 0? maybe not. This is why cloud computing is so popular, because we are not build one server, but we use a flexible architecture.
- For Free: a HUGE change in the business model, we have to find a different way to market our work. We must find different ways to get money from our work. Always remember: no longer a good but a service.
- Pay: "You don't pay, you don't play". In-game payment and DLC are a must today, microtransaction business model. We must make our users pay. We need a borker, because we are not a bank beacuse taking the money ourself is extremely difficult, service like paypal or visa do this or us. And also we have to managing sensible data? this is the reason why in several games we can register with Google or Facebook profile. Simply put, we cannot breakeven with just the sales ... not with AAA game. It's impossible for the companies to cover the cost of producing game only with the revenue from selling games, expecially if in the game we have like disney fee, or a famous actor. AAA company are trying to build the favourite pass time for people because in this case people change the value of the product and they can spend money on it.
- Online service: is very important point beacuse the game is no longer (only) a software, they are an online services. The online service you provide is an integral part of your project, when you shutdown the server the game is useless (or crippled at leas).

8.3 How to get money from the users

In a game usually there is an end game, but if we mean a game like a service we want to keep players playing.

We can have several situation:

- You are too greedy
- you sell too many copies
- You have too many users
- Your system is busted
- You are too greedy
- Your users get upset
- Your game is deserted
- You are too temperate, we asked few money and we are underbudget for developing the game
- You do not get enough users
- Your business is busted

We must look to the online thing as a component of our game:

- I did not say resource
- I did not say feature
- I implied the word strategic

Online will be a non optional part of our game and you must design your game around it.

- Do not use it occasionally
- Do not make it optional
- The way you will use the online component might determine success and failure of your game

This is everything about understanding if online is in or out of your core business:

- How deeply it integrates with your game
- How it relates with game mechanics

8.4 What do we use for it?

We have an universe of online games, we have a section of games that have online part as an option, in this games we can play offline but we also can play online (Minecraft, Gears of War). Then there is the rest of the game where online is designed as a feature, in many case this games can be played alone but it is not fun. Online experience providers give us the possibility to serve online as an added value.

The turning point to online as an option to a feature was Halo 3. The campaign is long and boring, but they want the player to play online the more amount of time, they also put a long campaign because in that time people want buy the box.

8.4.1 Another perspective

We have online capable games that can use online as a feature or as an option (we don't care much when we design an architecture), wrapped outside there is an infrastructure for online.

The added value infrastructure can also be seen as a middleware wrapping our game and acting as a bridge toward the outside.

The PRO is that our game will be protected, we don't need to care about what is outside. The CONS is that our game will be constrained, we cannot do anything without the infrastructure permission. I put you online I decide the rules.

The player is in the middle but he never asked to be protected from the outside.

We have data flow coming in and out. A channel that goes in is receiving advertising, that is the only way to pay a lower bill.

And of course there is a data flow going and going back from online gaming services.

And now we know why we cannot play online without paying a fee, because all these services have a cost.

8.5 It's a SERVICE

Under this prospective online gaming is no longer a software product, is a service! You are not just required to BUILD it you have to RUN it! It's a game?

- It's a SERVICE
- It's a WORLD
- It's a COMMUNITY

Anyone who says it's only a game is not catching the point.

It's more than a TelCo service, they have some common aspects:

- Manage HUGE servers
- Provide 7/24 assistance
- Must prevent frauds
- Host service from third parties

But TelCos have also different aspects:

- Do not produce software
- Do not produce content
- Do not need a community, just data flows
- Can assume all customers are overage
- Know customer the same as the payee

8.6 What is a Game Engine?

The story of GE is kind of complex and long now, we started to talk about good game architecture with Doom in 1994. Strict separation between core software components. Strict separation between software and data assets ... (this are the base for every project).

Other games have been designed with a modular architecture allowing modding and focusing on code reuse. Scripting language is part of the business...

What is a Game Engine: A software designed to run that simulation. Interface is not part of the deal in this case. Unreal and so on are interfaces of game engine implementation.

- Real time
- Agent based: we have independent entities agents that live and interact with each other within the engine
- Simultaion: It is capable to describe a model representation of a virutal world. It must be a mathematical description.

In Unity we can recognize all this stuff. Switching to an engine for an other is just learning a new interface, functionality are the same.

The idea is that game engine making our game run is a very small piece of code in charge to manage the basic simulation. Is like a micro kernel system. We call it CORE, it is about Resoruce management and cordination.

Around our corse sistem we ha evolution rules, the system has to change in time, and it follow the evolution rules.

Core is focused on performances. It must be able to apply any kind of rule.

We have the core in a blackbox, somethimes we know how the blackbox is build (like unreal that is open source). Sometimes we don't know anything about this box. The good thing is that we don't care about the black box. Unity is a cool graphical interface on top of the blackbox, and we use this GUI for everything. Other external stuff on top of the box are called tool set:

- To compile code
- To help you describe rules
- To manage assets
- To create content

The tool set couldn't be part of our game.

Runtime:

- Library/middleware/sandbox/virtual machine
- This will run your rules on your assets
- MUST be distributed with the game

The GUI is just the front end of the tool set.

Building a game is just filling up black box with assets and then ask it to create an executable files where my rules are applied to my assets.

All we need to understand is how the black box is working. As a matter of fact, it is not required for a game engine to have a GUI. Like Source from Valve.

A game engine is a container for RULES:

- You explain how the world is evolving. You explain the gravity, the effect to bouncing...
- You do not create the code to make it evolve

The black box will follow the rules and apply them on the assets you provided.

8.6.1 Asset

An asset is whatever you may think about to show, listen, or feel while playing.

- Texture
- 3D meshes
- Material definition
- Particles
- Visual effects (shaders)
- Music
- Scraps of code: because is text, is not a rule until we compile it and we attach it to an other asset

8.6.2 Rule

A rule is definition of behavior you attach to an asset in order to define:

- How to interact with the user
- How to interact with the environment
- How to interact with other assets

Creating a believable NPC means creating the "right" rules based on the surrounding context.

We usually create rules by writing code.

There are two kind of rules:

- Built-in: lighting, focusing on performances
- Provided by the user: this is what make our game truly unique. For example a particolare gravity. This is what we can call gameplay programming

There is no actual difference between the two. We can change built-in rules if we wish.

8.7 Runtime architecture

This is how to create a simple runtime architecture for game engine. It's quite scary.

Usually as developer we don't put our hand on game engine, there are specific developer that are dealing only with this, be aware that if we are this kind of developer we don't see the game.

Are very difficult to debut. Today if we are developer today we have to add stuff to the engine, if we are designer we have to write also the code.

A game engine is:

- Huge: usually MB of space
- Complex: because there are many pieces talking to each other
- Made of layers: like an onion or like a kernel of OS

Let's try again to define the architecture:

- Hardware
- Device drivers: to use the hardware and they are included in OS
- Operating System: it's not the same on PC (unless if we have xbox)
- Third party SDKs and middleware: some company bringing software that can be used in videogame
- Data Structures, Graphics, Physics, Animation, A.I. This is focused on performances and depends on the operating system and hardware
- Platform Independence layer: we want to create a game and export from different OS. Here all the assets and the rules are compiled. In this layer we have the same block but in more open and beautiful interface
- Core system
- Resource manager: manage what core system want to use
- Game virtual machine: my game is constraint in a sand box. In here we have rules like interaction, how to define output, network, debugger, compiler. That is basically the tool set. This is where the top of our black box is. Our game is outside of this.
- Game specific subsystem: there is where we define our own rules that are used by the core.

All we have to do is using the tool set in the top layer of this architecture. If we want to implement some network rule we build it on top of TCP RULES and OSI stack.

Three place where are rules.

There is an editor.

Some pieces are bundled with our game executable.

Games are compiled and stored in data structures layer. We pass the rules to executor and executor provide some output to the top.

A word about graphics. The loading screens are here because we are hiding while the assets are transforming to the right format. From Blender we import assets to Unity that have to translate this format to a platform independent format, this format have to be translated in engine specific formats, than this format has to be transformed in base of which library in our system is managing graphics, and the this library has to translate the asset to be used from graphic card.

There is no strict definition of engine modules:

- The rendered might know how to render a full fledged ogre or may just provide basic functionality
- The network manager may implement

8.8 Game Networking

We have client in the mibble internet and we have a server that provide the games. Of course we have several players. Remember that even if the clients are sharing the same server it does not mean we are playing together. Having a multiplayer game

8.9 Unity Networking

Is component based, we have component in game object that we want to be synchronized between client and server. We have to write code about the game and not about the network, because Unity will think about network.

We now have a package that we'll install through the package manager (Netcode for GameObjects).

8.9.1 UNet Architectures Overview

We have APIs governing game objects. Inside we have transport layer API, those are inside other API that are controlling the networked state of the game.

We use Game level API.

High Level API is controlling networking state of the game. A client will connect to a server running on the PC of another client. P2P architecture. Send s and receives stuff from e to someone.

8.9.2 Netcode Transport Architecture Overview

We have 3 layers, with 2 API's in the middle. In the Connection layer we can manage telemetry, channel handling, flow & congestion control. Telemetry is about controlling us.

The notification layer is the layer that send message to e from game object.

The driver layer is exchanging data between client and server. It use optimized UDP protocol with optimized payload. They also use multi channel design to avoid head of line blocking.

Support for quality of service.
Are flexible and dynamic to network topology.

8.9.3 Rules of the games

- Each GameObject has a representation on every connected clients. Exception: you may define a field of existence based on proximity for sake of performance, or to implement a field of vision of war. We don't want to transmit to client something that is not required.
- Each GameObject belongs to a client which is authoritative for that object. By default, server has authority on everything which is not a player
- All clients can modify directly only GameObject for which they have authority. This prevents cheating by implementing strict access control
- Modifications made on each client are pushed to the server and then propagated back to all other client.

How to apply the rules:

- we can assume all clients are using the same assets: maps, object, shaders, animations ... everything!
- We define networked only those objects that contribute to the shared experience. All other objects will be local and behave in the same way just because all assets are the same.

The rule is never, ever for any reason let a decision on the client! because requires more resources, increase lag and makes cheating possible.

The best way to do this is make the client authoritative only for empty data, like position, not the car. We send to the server our new position and the server move the car and calculate if we are crashed. This is why we have lag in online games.

This is cheat-proof, this approach is much more complicated to implement. For our prototype use the easy approach.

8.9.4 What do we need to sync over the network?

- GameObjects position and rotation. NetworkTransform
- Animation state for animated gameobjects. NetworkAnimation
- Generic values: elapsed time, energy left, free resources. You will tell Netcode to take care of this. NetworkVariable generic for fields. Or NetworkList generic for classes.

8.9.5 Spawning Gameobjects

In single player you Instantiate() an object.

In multiplayer you Spawn() and (already instantiated) network object.:

- You ask the server to replicate it on every client
- Remember to put it in the spawnable list
- Spawned object MUST have a NetworkObject component

8.9.6 How do we set up a game

- Create a Network Manager
- Create a visual interface to a lobby
- Networked scripts

Network Manager

Put it somewhere, in any gameobject which is not networked. You must have one. You can have only one. Note that extending NetworkManager is discouraged in Netcode.

8.9.7 Scenes

The lobby is one scene, the lobby is an other scene. When you we do our build we must drag and drop all scene assets here. And we are supposed to put the lobby's scene first.

8.9.8 Using networkManager

NetworkManager by itself is doing nothing. This is why we need an interface, because it's calling the methods to activate the network manager. To access NetworkManager we have to use singleton method.

8.10 Online Gaming Infrastructures

If it is a Service there is a Server. In online gaming infrastructure we have a piece of software in which we run the game and there is a game server that is not about the game. The game server is providing a service so we have data flow going in and out, the question here is which is the service? it's not about the game (could be about matchmaking...). In order to play we need to build between client and server a channel which must be secure, and be unavoidable (to control the players).

The company providing us the added value infrastructure is NOT interested in providing the game. It simply has another business. So we have not a server but a game network. Many services that we can use inside a game network is a game server (so game server is a small piece of game network). Game server usually is a third part part, for example Ubisoft pay Sony to be in the game network of Sony.

The game company is a CUSTOMER of the infrastructure company, they are going to PAY to connect to the infrastructure. The game company will request services to the infrastructure:

- Authentication
- Leaderboards
- Matchmaking
- Socials

This is mandatory. The game companies are usually forbidden to do that by themselves.

The game company will deal with the player and NOT with the person. They will not know your name, just you alias, but this is extremely good for the game company (GDPR). The player are a person only for game network provider, because they charge us the bill for their services.

All game companies connected to the same game network will offer a uniform set of service to their players. This will help achieve a smoother player experience between games and increase fidelity (customer retention).

In the old time we had the client connected to the Internet and a game server. Between we had a firewall to filter the Internet traffic from client to the server. Game server is not enough because we need so other services for example:

- Directory and security server
- Database server

This servers are not supporting the facing the gamer. Between game server and these servers we must have an other firewall for security reason. How much HW we need to do that. This is a 3 layer architecture:

- Outside the world
- DMZ: in between zone, this is cold we can reinstall all the stuff
- Private Network

Nowadays is not changed alot because we now have:

- Client
- Game Server
- Game Network

Instead of having game server now we have game network fronted (like Steam, GOG). If we need a game server we can use it connected t the game network but outside the reach of the player, an other firewall of course.

8.10.1 Frontend and Backend

We have two tier and they must be separated:

- Frontend: welcomes clients, perform initial authentication, and converts messages to a common format understandable from the backend
- Backend: is all the rest without distinction between game server and game network. We don't make difference between game server and network here, is like a backoffice service such as authentication and data storage

Be aware, every backend could have a backend itself, it's like a matryoshka.

8.10.2 Game cluster

Today we have game cluster. The backend may be backing the whole game infrastructure, but it is hosting also systems dedicated to run games.

May be different, independent, incarnations of the same world, they are also called shards such as realms in World of Warcraft

May be different worlds. May be different subsections of the same world. The game design must know this feature because there are some points in which the player jump from a cluster to other.

May be completely different games.

What we call "game server" may be the cooperation of two or more game clusters.

8.10.3 Cluster architecture

In cluster we have sub-servers and they create a full mesh connectivity. All this sub-server must share information (how is online? the state of the world, how many times player dies). To provide this we need at least two DB:

- World database: provide the state of environment, evolution of the world
- Character database: this is about the puppets (realm, class, inventory, name...)

8.10.4 Sub-server

May be independent section of a game world, this way we can increase computational and transmission capacity.

May be taking care of different map fragments this can be statically (game designer decide that one section is in one server, this usually is done looking the concentration of players in places) or dynamically.

May be taking care of different functions of the game.

8.10.5 Why clustering?

Why we are splitting so much our workload? easy, because we want to balance the workload:

- to increase the overall number of players
- To increase processing power:
 - To add detail to the worlds
 - To improve mobs A.I.

8.10.6 Workload

We may estimate workload in two ways:

- The number of players per sub-server:
 - Each server is assigned a fair share of the total players
 - Big overhead: players mostly move and interact
 - Data synchronization is HUGE problem
 - The number of players is not a good metric for many reasons. First and foremost the fact that not all the players behave in the same way
- The level of processing per sub-server
 - Each server is assigned a subsection of the world
 - Each server will take care of interaction and mobs in its area
 - Servers talk to each other only for (few) long-range issues
 - This solution is a VERY BAD option when you have seasonal or geographical events and all players converge on a single sub-server

8.10.7 Modern Age Clustering

Load balancing works different now. Now we have software entities inside huge cauldron which is the cloud. Yesterday we kept several servers and made sure to spread the workload evenly and pray that the total computational power was high enough. Today we switch on software servers when needed and pray that the total bill is low enough.

One of the most difficult operation today is to forecast cloud expenses.

8.10.8 High availability

Is the design for the system to be fault tolerant, not fault-proof a system can't be fault proof because a disaster can always happen.

Cluster balance load but may also provide hardware redundancy:

- Active passive cluster
- A machine is idle waiting for its sister to die
- Somehow, 50% of our money is wasted

Also remember: being idiot-proof is a completely different issue. Today, our hyper-scaler (cloud provider) will take care of this for us. Having information in cloud doesn't mean that information is safe, we have to backup our information even if they are on the cloud.

8.10.9 Business continuity

It's a different concept from high availability, business continuity is keeping the money flow running.

Is High Availability to keep system running plus a recovery strategy if we have fault of HW plus a contingency plan and sometimes also a disaster recovery site.

8.10.10 Make or buy?

Do NOT reinvent the wheel! If something is already there working and free (or cheap), use it.

Make an effort and use it, do not try to always create "better" code. Because our code will never be better than a existing commercial product.

This implies LOOKING for existing solutions or your problems! In the technical design document I will explicitly look for comparisons with existing solutions.

Chapter 9

Games and Network Interactivity

We have to talk the relationship between the game performance and the network.

9.1 Network Quality of service

Was a argument popular in the 80s. It's the capability of a network to provide better service to selected network traffic over various technologies. We developed technologies, HW (switches, network in which we create a physical infrastructure between you and destination) and Software (resources reservation, DLS). What we have today is MPLS used to simulate a network over an other network, putting a label on package and routers look for the package and forward the message. It's also true that we have only a game we don't need a lot of constraint. But the problem is that we need a lot of bandwidth to deliver a lot and at the right timing.

The world record for move per second is 818 that means that we have to give a feedback every 73 ms.

9.1.1 For UX

Network QoS must be able to deliver an enjoyable experience for overall feeling the player is perceiving from being in the flow, must be good enough to make me LIKE the game despite delay and packet loss.

9.1.2 Origin

Have been born like a dream in the late '90:

- No definitions
- No parameters
- No infrastructure

Was made for videoconference through Internet. At the time we have to guarantee network performance to a user given some media requirements. What we do today is overprovisioning, in the hope that will be fine.

9.1.3 Today

A guarantee about a data flow to have:

- A minimum bandwidth available
- A maximum end-to-end delay
- A uniform inter-arrival distribution (no pop corn effect)
- A maximum rate for packet loss

If we have these we are fine, but can we set this for a game? we have to define these over a long-haul connection while crossing several unknown domains over the Internet.

9.1.4 How to?

Technical solutions are available:

- Int-serv
- Diff-serv

A bunch of solutions to satisfy the user's hunger for a continuous and steady data flow:

- Infrastructure: give a separate infrastructure like in telecommunication
- Algorithms: how we deliver the package
- Strategies/Best practices
- Protocols

9.1.5 Let's Face It

Network QoS never worked for games (or any other application):

- Int-ser was not scalable enough
- Statistical quality of service of diff-serv is not acceptable

We must work with the resources we have!

The bubble of QoS exploded when we realized that o many cheap resources available, overprovisioning became the final answer to all questions.

9.1.6 Overprovisioning is NOT the answer

It is just a trap for the fools, because all your telco is giving you is more bandwidth. Having a huge network pipe means that we can send a lot of stuff at the same time, it does NOT mean that the stuff will reach the destination sooner!

Again the problem is not the bandwidth but the transmission delay.

To have data delivered in time, we must:

- Bound transmission delay
- Ensure jitter (instant variation over the average) over the connection is close to zero

There is NO WAY to be sure of that on today's networks.

Why do I play better if I have FTTH at home? Because fiber using ATM + SONET/SDH protocols provides an isochronous transmission.

9.1.7 QoS Today

We have local area network inside our house and then there is a connection to ISP router (the name of the protocol is PPOI point to point over internet because the connection between routers are point to point). Then the telco infrastructure reach the game infrastructure.

Where is the delay generator?

- Congestion in routers
- Retransmission: where a packet is lost we retransmit it

With modern network it's extremely difficult to lose packet during transimssion. So the delay is always due to congestion, if we kill the congestion, we can be able to reduce the delay.

Congestion may be on the Internet or in our infrastructure. It's rare that in the Internet we have congestion because they overprovisioning a lot, for this point of view Internet is an empty highway. Only one point remains, our house, Internet nowadays remain a empty highway with congested entry point. But why:

- Too much incoming traffic from the Internet:
 - Streaming (youtube, twitch, netflix...)
 - Data sync (GDrive, OneDrive, DropBox...)
 - Video calls
 - Mail
- Too much outgoing traffic from the LAN (big problem):
 - Data sync
 - Video calls
- Traversal traffic clogs your ISP link because there are too many active users in you area:
 - This prevent TCP acks from coming back
 - and you retransmit after a timeout
 - and you increase the end-to-end delay

None of this is related to gaming.

There is an other problem, the REAL problem. I have my setup and I know that the problem is there, link is not letting your packets going out at the rate you need.

The ISP solution is to pay more for more bandwidth. Always remember the distinction between network bandwidth and network capacity! bandwidth is spectrum, is about the range of frequency that we can use to transmit, the network capacity is the amount of bits that can traverse our media in range of time.

Then how do we make people pay?

- Easy solution: Overprovisioning

- Easy to deploy: It's already there
- Easy

In order to overprovisioning we have to own the network and owning a WAN is an IMMENSE cost.

9.1.8 It is NOT like that

Experience is a matter of quality, not quantity! Games today are designed to use bandwidth sparingly, to save on the (provider) bill. We don't need to send much over the internet, all assets are local.

Ongoing research is currently focusing on three subjects:

- Understanding how network unfairness effects gameplay
- Expanding games scalability: the next frontier for gaming is having 1000s players together
- Games inter-operability

9.1.9 Long Term Bandwidth Usage

This is called patch distribution, as in download capability. You cannot distribute a 150+ MB patch to be downloaded. This is not a technical problem rather than an organization problem.

9.2 Latency and Gaming

We assume that delay will be that much as we design our mechanics in order to stand that delay.

- DLS: 60ms
- Dial-up: 200ms
- GPRS: seconds
- UMTS: hundreds of ms
- LTE (4G): hundreds of ms
- 5G: nearly nothing

9.2.1 5G

5G is a transition technology. We had 2 different words:

- Sensor that can collect data but not elaborate it
- Cloud that have almost infinite computational power

We need sensor data jumping in the cloud. 5G reduce the delay to reach the access point, but after that there is Internet and we'll acquire delay. If we make computation on the access point we can have instant replay, but it's not what happens right now. 5G is something in progress, with the version 18 we started to deploy services on the edge (like zoom calls).

9.3 How we deal with delay in games

As a matter of fact, when we play, we care only about LAG:

- Ever wondered there is no “bandwidth hog” indicator in any client?
- Even measuring it is not very straightforward

Usually, we feel happy (and fair) if all players suffer from the same “acceptable” delay:

- LAG compensation techniques can be applied: the delay of all connection is made even with the worst one
- The definition of “acceptable” depends on the kind of game you play

If the LAG is (fairly) increased (too much) for everyone the game may be perceived as playable, but we might see some nasty side-effects.

9.3.1 How to obtain a uniform inter-arrival

Heaven: one frame update in each packet and one packet exactly every 1/60th of second. Too good to be true!

In reality this never happen. So we have to handle with jitter.